Web and Data Engineering

Lecture 04: HTTP und RESTful APIs

Prof. Dr. Michael Granitzer



Ziel dieser Vorlesung

HTTP ist das Protokoll des Webs. REST nutzt dieses Protokoll konsequent als Grundlage für API-Design.

> Diese Vorlesung verbindet beides: vom **Protokollverständnis** über **Caching und Sicherheit** bis zum **Entwurf robuster, evolvierbarer APIs**.

Leitfragen

- Warum ist HTTP mehr als Datentransport?
- Wie strukturieren Header HTTP-Nachrichten und ihre Repräsentationen?
- Wie steuern Header Caching, Sessions und Sicherheit?
- Was unterscheidet REST von beliebigem JSON-über-HTTP?
- Wie modelliert man Ressourcen, URIs und HTTP-Semantik für APIs?
- Wie entwickelt man APIs weiter, ohne Clients zu brechen?

HTTP Grundlagen

Leitfrage: Wie können Browser und Server mit einem einfachen, textbasierten Protokoll so kommunizieren, dass das Web weltweit skalierbar bleibt?

Was sehen Sie hier?

Ein Nutzer ruft die Produktseite auf. Das Netzwerk-Tab zeigt diesen Austausch:

↑ REQUEST

```
GET /api/products/42 HTTP/1.1
Host: shop.example.com
Accept: application/json
Cookie: session=abc123
```

↓ RESPONSE

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: max-age=300
ETag: "f7e3a1b2"

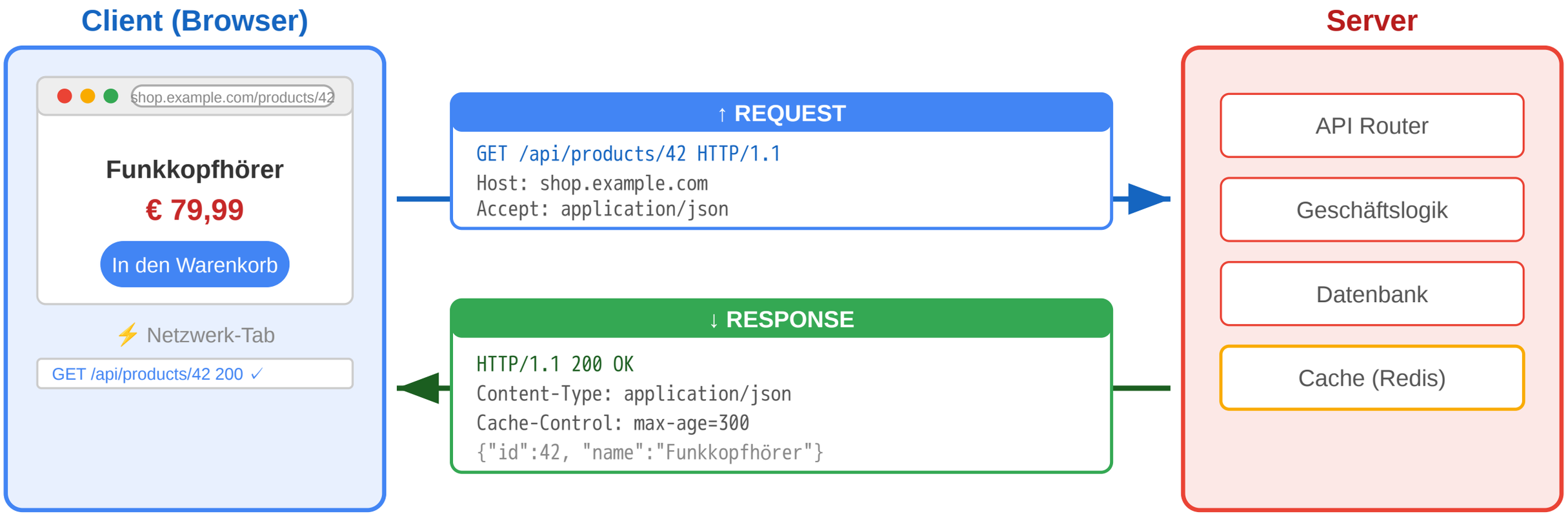
{"id":42,"name":"Funkkopfhörer","price":79.99}
```

Aufgabe: Was ist Startzeile, Header, Body? Welche Information steckt in jedem Header?

Teil	Request	Response
Startzeile	`GET /api/products/42 HTTP/1.1`	`HTTP/1.1 200 OK`
Header	Host, Accept, Cookie	Content-Type, Cache-Control, ETag
Body	kein Body bei GET	JSON-Daten des Produkts

Cache-Control: max-age=300
Daten 5 min gültig
ETag: "f7e3a1b2"
Fingerprint zur Validierung

HTTP: Das Request-Response-Protokoll



Dazwischen: DNS · CDN/WAF · Load Balancer

Von der Fetch API zum Protokoll

In Vorlesung 03 haben wir die **Client-Seite** gesehen:

```
const response = await fetch('/api/products/42', {
  headers: { 'Accept': 'application/json' }
});
```

Jetzt schauen wir auf die **Protokollseite**:

Fetch API	HTTP-Protokoll
`fetch(url)`	Browser erzeugt HTTP-Request mit Startzeile + Headern
`{ method: 'POST' }`	HTTP-Methode — definiert Absicht und Semantik
`{ headers: {...} }`	Request-Header — Metadaten für Server und Infrastruktur
`response.status`	Statuscode — standardisiertes Ergebnis
`response.json()`	Response Body parsen — Repräsentation der Ressource

> Die Fetch API ist eine **JavaScript-Abstraktion über HTTP**. Wer HTTP versteht, versteht auch, warum `fetch` so funktioniert wie es funktioniert — und kann Netzwerkprobleme im DevTools-Netzwerk-Tab diagnostizieren.

HTTP-Designprinzipien

Prinzip	Bedeutung	Konsequenz
Ressourcenorientiert	Jede Information hat eine Adresse (URI)	Einheitliche Adressierung weltweit
Request-Response	Client fragt, Server antwortet	Klare Rollenverteilung
Zustandslos	Jeder Request ist unabhängig	Server muss keinen Kontext halten → skaliert
Erweiterbar	Header erlauben beliebige Metadaten	Caching, Auth, Content-Negotiation ohne Protokolländerung
Textbasiert	Nachrichten sind menschenlesbar (HTTP/1.1)	Einfach zu debuggen

Warum das skaliert

- Zustandslosigkeit erlaubt Load Balancing über beliebig viele Server
- Proxies und Caches können HTTP-Nachrichten ohne Anwendungswissen verarbeiten
- Klare Trennung von Transport und Anwendungslogik

URI (Uniform Resource Identifier)

Schema

`scheme://host:port/path?query#fragment`

Example

`https://shop.example.com:443/api/products/42?lang=de#details`

Teil	Beispiel	Bedeutung
Scheme	`https`	URI-Schema vor `:`; bestimmt, wie die URI interpretiert wird
Host	`shop.example.com`	DNS-Name des Servers
Port	`443`	TCP-Port; bei HTTPS implizit 443
Pfad	`/api/products/42`	Adresse der Ressource auf dem Host
Query	`?lang=de`	Parameter für Auswahl, Filter, Darstellung
Fragment	`#details`	clientseitiger Sprunganker; wird nicht an den Server gesendet

HTTP-Methoden als semantischer Vertrag

Methode	Absicht	Sicher?	Idempotent?	Online-Shop-Beispiel
GET	Ressource lesen	ja	ja	Produktseite anzeigen
POST	Neue Ressource oder Verarbeitung	nein	nein	Bestellung aufgeben
PUT	Ressource vollständig ersetzen	nein	ja	Profil aktualisieren
PATCH	Ressource teilweise ändern	nein	nein*	Adresse ändern
DELETE	Ressource entfernen	nein	ja	Konto löschen
HEAD	Nur Header, kein Body	ja	ja	Existenzprüfung
OPTIONS	Erlaubte Methoden abfragen	ja	ja	CORS Preflight

Sicher (Safe): Methode verändert keine Ressource — kann ohne Nebenwirkungen wiederholt werden. CDNs cachen nur safe Methoden.

Idempotent: Mehrfache Ausführung hat denselben Effekt wie einmal — wichtig für Retry-Logik in Load Balancern und API-Gateways.

Achtung: Die Unterscheidung ist eine **Konvention**, keine technische Durchsetzung. Jeder Server kann jede Methode für jede semantische Bedeutung nutzen. Die Tabelle beschreibt die **übliche** Interpretation.

HTTP-Methoden: Safe und Idempotent in der Praxis

Szenario: Ein Nutzer klickt zweimal schnell auf „Jetzt kaufen“. Netzwerk-Lag verzögert den ersten Request, beide kommen beim Server an.

Methode	Effekt beim Doppel-Request	Warum?
`POST /api/orders`	**Zwei Bestellungen** entstehen	Nicht idempotent — jeder Aufruf erzeugt neue Ressource
`PUT /api/cart/item/42`	**Eine Änderung** — zweiter Call ohne Effekt	Idempotent — gleiches Ergebnis egal wie oft

Konsequenz: Load Balancer und API-Gateways können idempotente Requests bei Timeout sicher wiederholen. CDNs cachen nur **safe** Methoden (GET, HEAD), weil diese die Ressource nie verändern.

HTTP-Statuscodes

Klasse	Bedeutung	Wichtige Codes
1xx	Information	`101 Switching Protocols`
2xx	Erfolg	`200 OK`, `201 Created`, `204 No Content`
3xx	Umleitung	`301 Moved Permanently`, `304 Not Modified`
4xx	Client-Fehler	`400 Bad Request`, `401 Unauthorized`, `403 Forbidden`, `404 Not Found`, `429 Too Many Requests`
5xx	Server-Fehler	`500 Internal Server Error`, `502 Bad Gateway`, `503 Service Unavailable`

Was stimmt an dieser API-Antwort nicht?

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "success": false,
  "error": "Produkt nicht gefunden"
}
```

Problem: Statuscode lügt

- `200 OK` signalisiert Erfolg
- Body enthält aber einen Fehler
- CDNs cachen die „Erfolg“-Antwort
- Monitoring zählt 200 als OK

Richtig: `404 Not Found` als Statuscode, Fehlerdetails im Body.

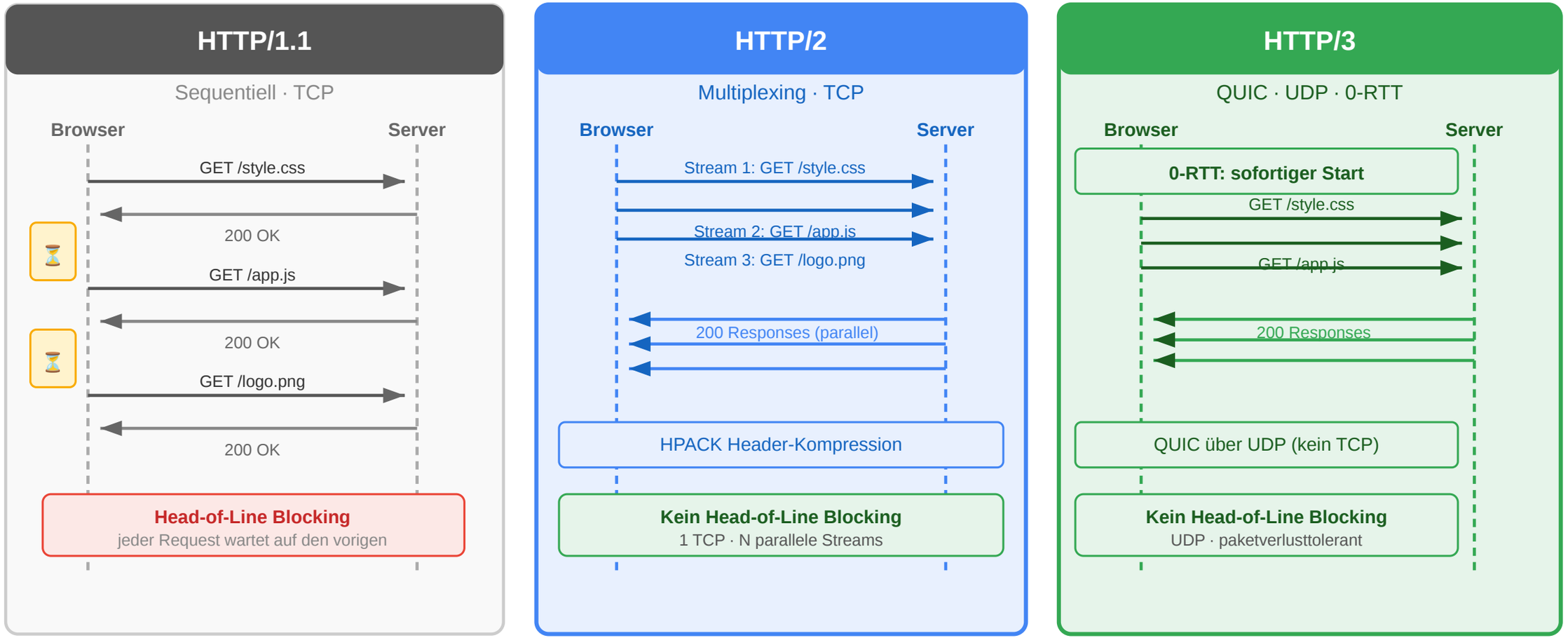
HTTP-Versionen im Vergleich

Version	Jahr	Kernverbesserung
HTTP/1.0	1996	Ein Request pro TCP-Verbindung
HTTP/1.1	1997	Persistent Connections, Pipelining
HTTP/2	2015	Multiplexing, Header-Kompression, binär
HTTP/3	2022	QUIC (UDP-basiert), schnellerer Aufbau

QUIC ist der Transport unter HTTP/3: Es läuft über UDP, integriert Verschlüsselung direkt und kann Verbindung plus TLS schneller aufbauen. Wichtig: QUIC ersetzt hier TCP, HTTP-Semantik wie Methoden, Statuscodes und Header bleibt erhalten.

HTTP ist nicht nur Datentransport — es ist das **Interaktionsmodell** des Webs. Methoden definieren Absichten, Statuscodes kommunizieren Ergebnisse, Header steuern Verhalten. Infrastruktur (CDN, Load Balancer, WAF) verlässt sich auf diese Semantik. Wer HTTP als Syntax behandelt, verliert die Vorteile des Protokolls.

HTTP-Versionen im Vergleich



Frage: Eine Produktseite lädt 60 Ressourcen (JS, CSS, Bilder). Warum dauert das mit HTTP/1.1 deutlich länger als mit HTTP/2 — obwohl die Bandbreite dieselbe ist?

HTTP Headers and Status Codes

Leitfrage: Wie strukturieren Header und Statuscodes HTTP-Nachrichten, sodass Browser, Caches und Anwendungen ohne fachliches Vorwissen entscheiden können, was zu tun ist?

HTTP-Header im Überblick

Eine HTTP-Nachricht besteht aus:

1. **Startzeile**
   Request: Methode + Ziel + Version
   Response: Version + Statuscode + Statusmeldung
2. **Header-Felder**
   Metadaten als `Name: Wert`
3. **Leerzeile**
   Trennt Header und Body
4. **Optionaler Body**
   Die eigentliche Repräsentation

```
GET /products/42 HTTP/1.1
Host: shop.example.com
Accept: application/json
```

Wie sind HTTP-Header strukturiert?

Teil	Bedeutung	Beispiel
Feldname	Standardisierter Header-Name	`Content-Type`
Doppelpunkt	Trennt Name und Wert	`:`
Feldwert	Semantik des Headers	`application/json`

```
Content-Type: application/json; charset=utf-8
Cache-Control: public, max-age=3600
Authorization: Bearer eyJhbGciOi...
```

- Header-Namen sind **case-insensitive**
- Ein Feldwert kann aus **Token, Parametern oder Listen** bestehen
- Viele Header haben eine klar definierte **Syntax nach RFC**

Kategorien von Header-Feldern

Kategorie	Typische Header	Zweck
Request-Header	`Host`, `Accept`, `Authorization`, `Cookie`	Beschreiben den eingehenden Request
Response-Header	`Server`, `Location`, `Set-Cookie`	Beschreiben Eigenschaften der Antwort
Repräsentations-Header	`Content-Type`, `Content-Length`, `Content-Encoding`, `Content-Language`	Beschreiben den Body
Caching / Conditional	`Cache-Control`, `ETag`, `If-None-Match`, `Last-Modified`	Steuern Wiederverwendung und Validierung
Sicherheits-Header	`Strict-Transport-Security`, `Content-Security-Policy`	Erhöhen Sicherheitseigenschaften

Wichtig: Dieselbe HTTP-Ressource kann mehrere Repräsentationen haben. Die Header beschreiben, **welche Variante gerade übertragen wird**.

Typische Request- und Response-Header

Header	Typ	Kurz erklärt
`Host`	Request	Gibt an, für welchen Host der Request bestimmt ist; wichtig bei virtuellen Hosts
`Authorization`	Request	Übertraegt Anmeldedaten oder Zugriffstoken
`Location`	Response	Verweist auf die URI einer neuen oder anderen Ressource
`Server`	Response	Kann Informationen über die Server-Software enthalten

```
POST /login HTTP/1.1
Host: shop.example.com
Authorization: Bearer eyJ...
```

Typische Header für Repräsentation und Kontrolle

Header	Kategorie	Kurz erklärt
`Content-Length`	Repräsentation	Nennt die Länge des Bodys in Bytes
`Content-Encoding`	Repräsentation	Gibt an, ob der Body z. B. mit gzip komprimiert wurde
`Content-Security-Policy`	Sicherheit	Beschränkt, welche Inhalte eine Seite laden oder ausführen darf
`Strict-Transport-Security`	Sicherheit	Erzwingt für eine Domain die Nutzung von HTTPS

Merksatz:
Ein Teil der Header beschreibt den **Body**, ein anderer Teil steuert **Verhalten** von Clients, Caches und Browsern.

Was ist an dieser API-Antwort falsch?

Ein Client sendet eine Bestellung an den Online-Shop:

```
POST /api/orders HTTP/1.1
Content-Type: application/json

{"product_id": 42, "quantity": 2}
```

Der Server antwortet:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"id": 789, "status": "created"}
```

> **Aufgabe:** Die Bestellung wurde erfolgreich erstellt. Trotzdem hat die Response mindestens drei Probleme. Welche?

Was ist an dieser API-Antwort falsch?

Ein Client sendet eine Bestellung an den Online-Shop:

```
POST /api/orders HTTP/1.1
Content-Type: application/json

{"product_id": 42, "quantity": 2}
```

Der Server antwortet:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"id": 789, "status": "created"}
```

> **Aufgabe:** Die Bestellung wurde erfolgreich erstellt. Trotzdem hat die Response mindestens drei Probleme. Welche?

Problem	Warum	Besser
`200 OK` statt `201 Created`	`200` sagt „gelesen", nicht „erzeugt"	`201 Created`
Kein `Location`-Header	Client weiß nicht, wo die neue Ressource liegt	`Location: /api/orders/789`
Kein `Cache-Control`	Bestellungen dürfen nicht gecacht werden	`Cache-Control: no-store`

Korrigierte Version

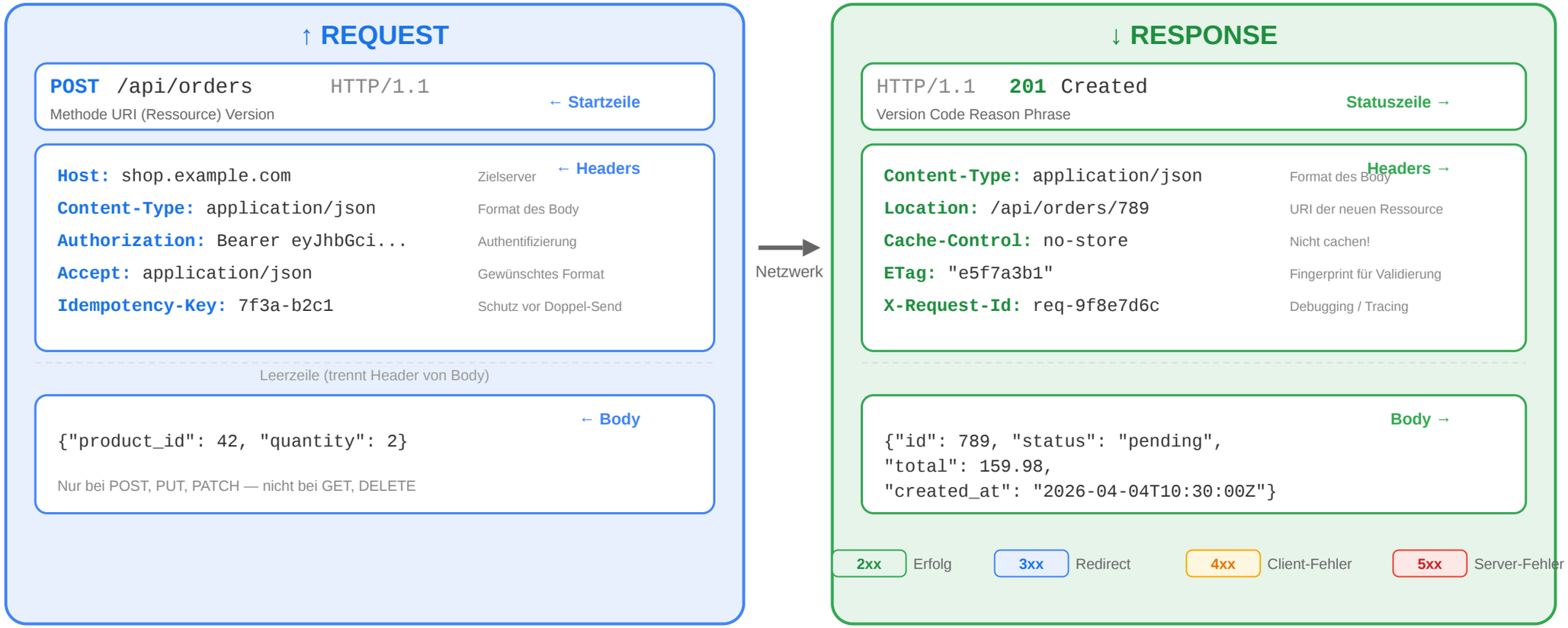


```
HTTP/1.1 201 Created
Location: /api/orders/789
Content-Type: application/json
Cache-Control: no-store
```

```
{"id": 789, "status": "pending", "total": 159.98}
```

Der vollständige Request-Response-Zyklus

Anatomie: HTTP Request und Response



Statuscodes richtig einsetzen

Situation	Richtiger Code	Häufiger Fehler
Ressource erfolgreich gelesen	`200 OK`	—
Ressource erfolgreich angelegt	`201 Created` + Location	`200` ohne Location
Erfolgreich, aber kein Body	`204 No Content`	`200` mit leerem Body
Ungültige Eingabedaten	`400 Bad Request`	`500` (ist kein Server-Fehler!)
Nicht authentifiziert	`401 Unauthorized`	`403`
Authentifiziert, aber nicht berechtigt	`403 Forbidden`	`401`
Ressource nicht gefunden	`404 Not Found`	`200` mit Fehlermeldung im Body
Konflikt (z.B. doppelter Username)	`409 Conflict`	`400`

Faustregel:
Der Statuscode sagt dem Client, **was passiert ist**. Der Body erklärt **warum** und **was zu tun ist**.

Fehlermodelle: Konsistenz ist alles

```
{
  "error": {
    "type": "validation_error",
    "message": "Die Eingabedaten sind ungültig.",
    "details": [
      { "field": "email", "code": "invalid_format",
        "message": "Keine gültige E-Mail-Adresse" },
      { "field": "quantity", "code": "out_of_range",
        "message": "Muss zwischen 1 und 100 liegen" }
    ]
  }
}
```

Prinzip	Umsetzung
Konsistente Struktur	Immer dasselbe JSON-Format für Fehler
Maschinenlesbar	`type` und `code` für automatische Verarbeitung
Menschenlesbar	`message` für Entwickler und Endnutzer
Spezifisch	`details` pro Feld bei Validierungsfehlern
Kein Stacktrace	Interne Details nie an den Client

Representations and Caching

Leitfrage: Wie liefert HTTP dieselbe Ressource in verschiedenen Formaten aus, und wie vermeidet man dabei unnötiges Übertragen?

Medienrepräsentationen und MIME Types

Eine Ressource ist nicht gleich ihr Datenformat. Dieselbe Ressource kann als verschiedene **Medientypen** übertragen werden:

Medientyp	Bedeutung	Beispiel
`text/html`	HTML-Dokument	Produktseite im Browser
`application/json`	JSON-Daten	API-Response
`text/css`	Stylesheet	Gestaltung einer Seite
`application/javascript`	JavaScript-Code	Skriptdatei
`image/png`	PNG-Bild	Produktfoto
`application/pdf`	PDF-Dokument	Rechnung zum Download

Media Type = Typ/Subtyp
Beispiele: `text/html`, `application/json`, `image/svg+xml`

Der Medientyp beschreibt nicht die Ressource selbst, sondern die Form ihrer aktuellen Repräsentation.

Verschiedene Clients, verschiedene Repräsentationen

Nicht jeder Client braucht dieselbe Darstellung derselben Ressource:

Client-Modell	Typische Repräsentation	Wofür geeignet?
Server-gerenderter Browser-Client	`text/html`	Der Server liefert direkt eine fertige HTML-Seite
JavaScript-Frontend / SPA	`application/json`	Das Frontend rendert die Oberfläche selbst
Mobile App	`application/json`	Die App verarbeitet Daten und baut die UI nativ
Download-Client	`application/pdf`, `image/png`	Datei- oder Medienausgabe

Kernidee:
Die fachliche Ressource bleibt gleich, aber die **Repräsentation** wird an den Bedarf des Clients angepasst.

HTML vs. JSON als Repräsentation

HTML-Repräsentation

- Enthält Struktur und Inhalt für die direkte Anzeige im Browser
- Typisch für **serverseitig gerenderte** Anwendungen
- Media Type: `text/html`

JSON-Repräsentation

- Enthält strukturierte Daten statt fertiger Seiten
- Typisch für **APIs**, SPAs und mobile Clients
- Media Type: `application/json`

Wichtig:
HTML und JSON konkurrieren nicht zwingend. Beide können sinnvolle Repräsentationen **derselben Ressource** sein.

JSON kurz eingeführt

JSON steht für **JavaScript Object Notation**. Es ist ein leichtgewichtiges Textformat für strukturierte Daten.

```json
{
  "id": 42,
  "name": "Funkkopfhörer",
  "price": 79.99,
  "in_stock": true,
  "tags": ["audio", "wireless"],
  "manufacturer": {
    "name": "Acme",
    "country": "DE"
  },
  "discount": null
}
```

JSON-Element	Beispiel
Objekt	`{ "id": 42 }`
Array	`[1, 2, 3]`
String	`"name"`
Zahl	`79.99`
Boolean	`true`, `false`
Null	`null`

Content Negotiation: Grundidee

Header	Richtung	Bedeutung
`Content-Type`	Request oder Response	Welches Format der Body hat
`Accept`	Request	Welche Formate der Client akzeptiert
`Accept-Language`	Request	Bevorzugte Sprache
`Accept-Encoding`	Request	Bevorzugte Kompression, z. B. gzip oder br
`Content-Encoding`	Response	Welche Kompression auf den Body angewendet wurde

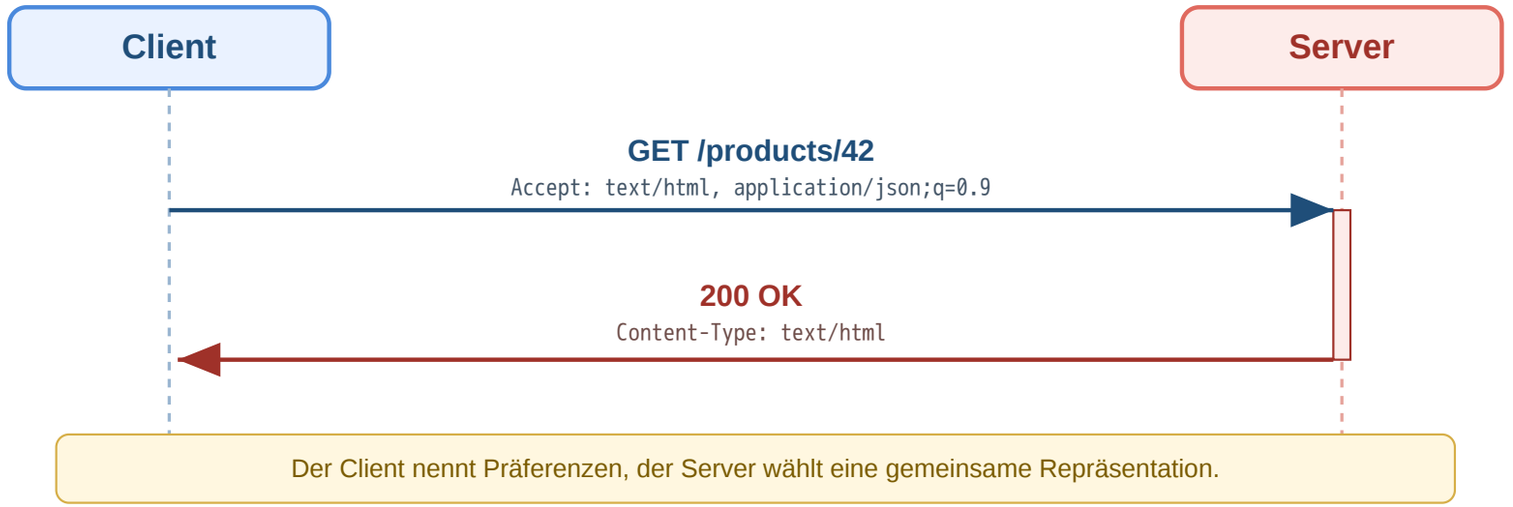
```
GET /products/42 HTTP/1.1
Accept: application/json
Accept-Language: de
Accept-Encoding: gzip, br
```

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Language: de
Content-Encoding: gzip
```



Content Negotiation: Repräsentation auswählen

Ein Client kann mehrere Formate akzeptieren und Prioritäten angeben:



Teil	Bedeutung
`text/html`	Bevorzugte Repräsentation
`application/json;q=0.9`	Ebenfalls akzeptabel, aber etwas weniger bevorzugt
`application/xml;q=0.5`	Nur niedrige Priorität

q-Wert = Qualitätswert
Je höher der q-Wert, desto stärker die Präferenz des Clients.
Wenn kein gemeinsames Format gefunden wird, kann der Server z. B. mit `406 Not Acceptable` antworten.

Eine Ressource, mehrere Repräsentationen

Beispiel: Dieselbe Produkt-Ressource `/products/42`

Browser-orientiert

```
GET /products/42 HTTP/1.1
Accept: text/html
```

```
HTTP/1.1 200 OK
Content-Type: text/html
```

API-orientiert

```
GET /products/42 HTTP/1.1
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json
```

Kernidee:
Die **Ressource** bleibt dieselbe, aber ihre **Repräsentation** kann je nach Client unterschiedlich sein.



Warum das für HTTP-Verständnis wichtig ist

- `Content-Type` und `Accept` machen Repräsentationen explizit
- Caches müssen wissen, **welche Variante** einer Ressource gespeichert wurde
- Infrastruktur kann nur korrekt arbeiten, wenn Header die Nachricht eindeutig beschreiben
- Dieselbe fachliche Ressource kann für unterschiedliche Clients unterschiedlich dargestellt werden

> Header sind nicht Beiwerk, sondern Teil der HTTP-Semantik. Sie beschreiben Formate, Aushandlung, Kompression, Caching, Sicherheit und Zustand. Ohne dieses Verständnis bleiben spätere Themen wie Content Negotiation, Cache-Control oder API-Design unvollständig.

Szenario: Der veraltete Preis

Ein Nutzer besucht die Produktseite des Funkkopfhörers. Der Preis wird von **€ 79,99 auf € 69,99 gesenkt**. Der Nutzer lädt die Seite erneut und sieht trotzdem weiter den alten Preis.

Aufgabe: Was ist passiert? Welche HTTP-Mechanismen sind beteiligt?

Szenario: Der veraltete Preis

Ein Nutzer besucht die Produktseite des Funkkopfhörers. Der Preis wird von **€ 79,99 auf € 69,99 gesenkt**. Der Nutzer lädt die Seite erneut und sieht trotzdem weiter den alten Preis.

Aufgabe: Was ist passiert? Welche HTTP-Mechanismen sind beteiligt?

> Der Browser hat die vorherige Response gecacht. Der `Cache-Control: max-age=300` Header sagte: "Diese Daten sind 5 Minuten gültig." Innerhalb dieser Frist fragt der Browser **gar nicht** beim Server nach.

Szenario: Der veraltete Preis

Ein Nutzer besucht die Produktseite des Funkkopfhörers. Der Preis wird von **€ 79,99 auf € 69,99 gesenkt**. Der Nutzer lädt die Seite erneut und sieht trotzdem weiter den alten Preis.

Aufgabe: Was ist passiert? Welche HTTP-Mechanismen sind beteiligt?

> Der Browser hat die vorherige Response gecacht. Der `Cache-Control: max-age=300` Header sagte: "Diese Daten sind 5 Minuten gültig." Innerhalb dieser Frist fragt der Browser **gar nicht** beim Server nach.

> **Architekturentscheidung:** Wie lange sollen Produktdaten gecacht werden? Zu lang -> veraltete Preise. Zu kurz -> jeder Seitenaufruf belastet den Server.

Caching: Ebenen und Mechanismen

Cache-Ebene	Wo?	Vorteil
Browser-Cache	Lokal auf dem Gerät	Kein Netzwerk-Request -> schnellste Variante
Proxy/CDN-Cache	Edge-Server im Netzwerk	Reduziert Last auf Origin, niedrige Latenz
Application-Cache	Im Server (Redis, Memcached)	Vermeidet wiederholte Datenbankabfragen

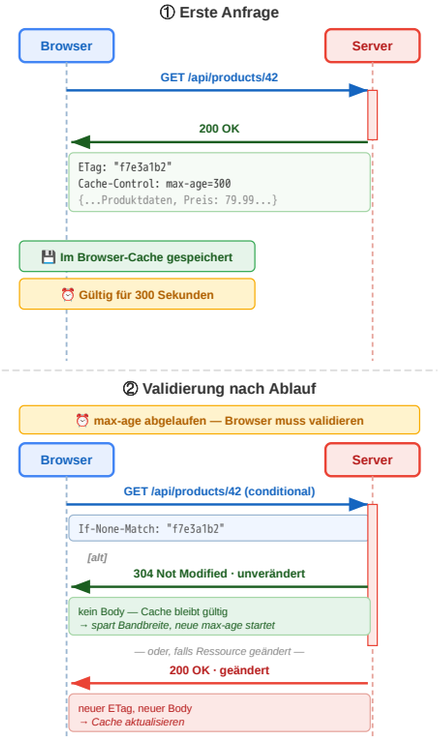
Jede Ebene spart Arbeit an einer anderen Stelle:

- Browser-Cache spart Netzverkehr
- CDN spart Origin-Last
- Application-Cache spart Datenbank- oder Rechenaufwand

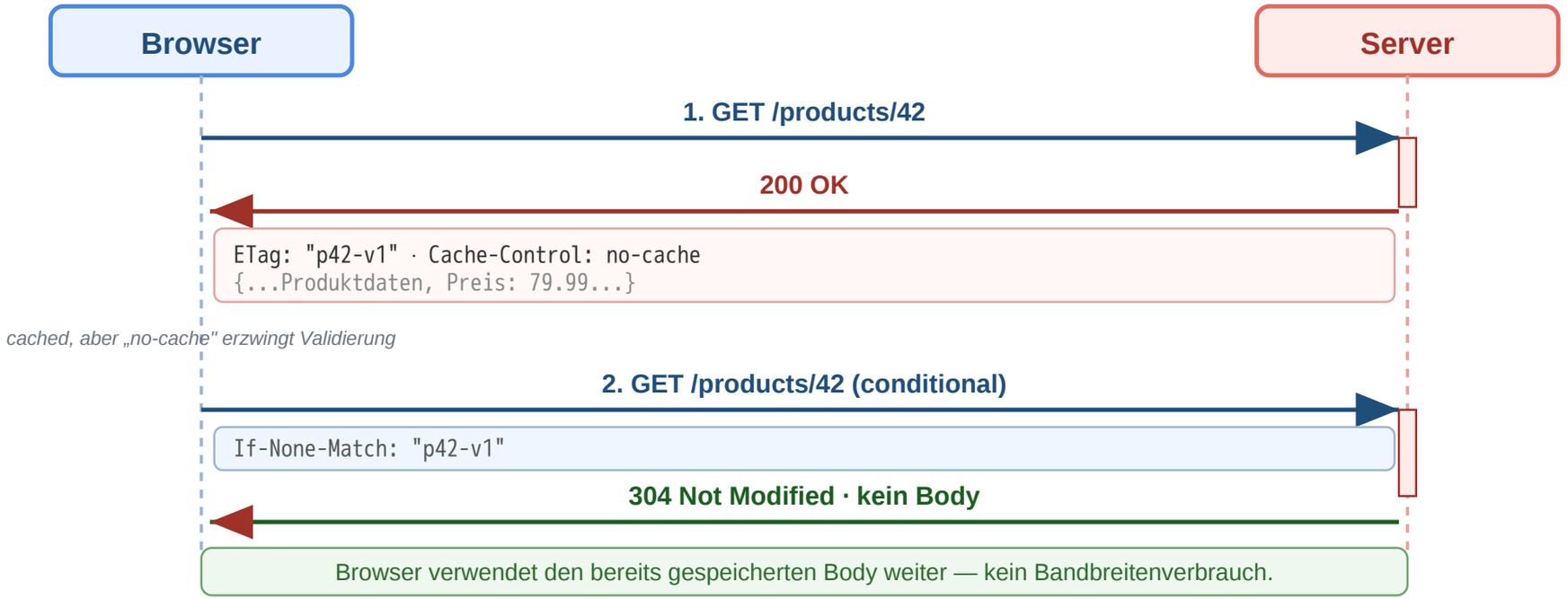
Cache-Steuerung über Header

Header	Funktion	Beispiel
`Cache-Control: max-age=3600`	1 Stunde gültig	Statische Assets
`Cache-Control: no-cache`	Immer validieren	Produktdaten, Preise
`Cache-Control: no-store`	Nie cachen	Bankdaten
`Cache-Control: public`	CDN darf cachen	Öffentliche Inhalte
`Cache-Control: private`	Nur Browser	Nutzerdaten
`ETag`	Ressourcen-Fingerprint	`"a1b2c3d4"`
`Last-Modified`	Letzter Änderungszeitpunkt	`Mon, 23 Mar 2026`

Wie der Browser entscheidet: Cache vorhanden? → max-age abgelaufen? → Validierungsrequest mit `If-None-Match: ETag`
Validierung: Server antwortet `304 Not Modified` (kein Body, spart Bandbreite) oder `200 OK` + neuer Body + neuer ETag.



Workflow: Conditional Request und Cache-Validierung



Schritt	Bedeutung
`ETag` empfangen	Client kennt Version der Repräsentation
`If-None-Match` senden	Client fragt: Hat sich etwas geändert?
`304 Not Modified`	Kein neuer Body nötig, Cache bleibt gültig

Was ist ein ETag? Ein **Entity Tag** ist ein vom Server vergebener, **opaker Versions-Fingerprint** einer konkreten Repräsentation. Format: in Anführungszeichen, z.B. `ETag: "a1b2c3d4"`. Der Client interpretiert ihn nie — er vergleicht nur.

Wie wird er berechnet? Die Wahl liegt beim Server; er muss nur garantieren, dass *gleicher Inhalt → gleicher ETag* und *geänderter Inhalt → anderer ETag*. Übliche Strategien:

Strategie	Beispiel	Wann sinnvoll
Inhalts-Hash	`MD5`/`SHA-1`/`xxHash` über den Response-Body	Statische Dateien, deterministisch generierte JSON-Responses
Metadaten-Kombination	`mtime + size + inode` (z.B. Apache, nginx)	Dateisystem-basierte Auslieferung — billig, kein Lesen nötig

Strategie	Beispiel	Wann sinnvoll
Datenbank-Version	`updated_at`-Timestamp oder `rev`-Spalte	API-Ressourcen aus DB — sehr effizient, kein Hashing

Stark vs. schwach: `"a1b2c3d4"` (strong) = byte-identisch; `W/"a1b2c3d4"` (weak, mit `W/`-Prefix) = semantisch äquivalent (z.B. Whitespace-Unterschiede irrelevant). Strong-ETags sind Pflicht für Range-Requests, weak reichen für Cache-Validierung.

Interpretationsaufgabe: Caching-Strategie

Aufgabe: Welchen `Cache-Control`-Header würden Sie für diese Ressourcen des Online-Shops setzen?

Ressource	Ihr Vorschlag?
`app.v3.2.1.js` (versionierte JS-Datei)	
`/api/products/42` (Produktdaten mit Preis)	
`/api/cart` (Warenkorb des eingeloggten Nutzers)	
`logo.png` (Shop-Logo)	

Interpretationsaufgabe: Caching-Strategie

Aufgabe: Welchen `Cache-Control`-Header würden Sie für diese Ressourcen des Online-Shops setzen?

Ressource	Ihr Vorschlag?
`app.v3.2.1.js` (versionierte JS-Datei)	
`/api/products/42` (Produktdaten mit Preis)	
`/api/cart` (Warenkorb des eingeloggten Nutzers)	
`logo.png` (Shop-Logo)	

Ressource	Header	Begründung
`app.v3.2.1.js`	`public, max-age=31536000, immutable`	Version im Dateinamen → ändert sich nie
`/api/products/42`	`public, no-cache` + ETag	Preise können sich ändern, Validierung nötig
`/api/cart`	`private, no-store`	Nutzerspezifisch, darf nicht im CDN landen
`logo.png`	`public, max-age=86400`	Ändert sich selten, 1 Tag reicht

Redirects und das Post/Redirect/Get-(PRG)-Pattern

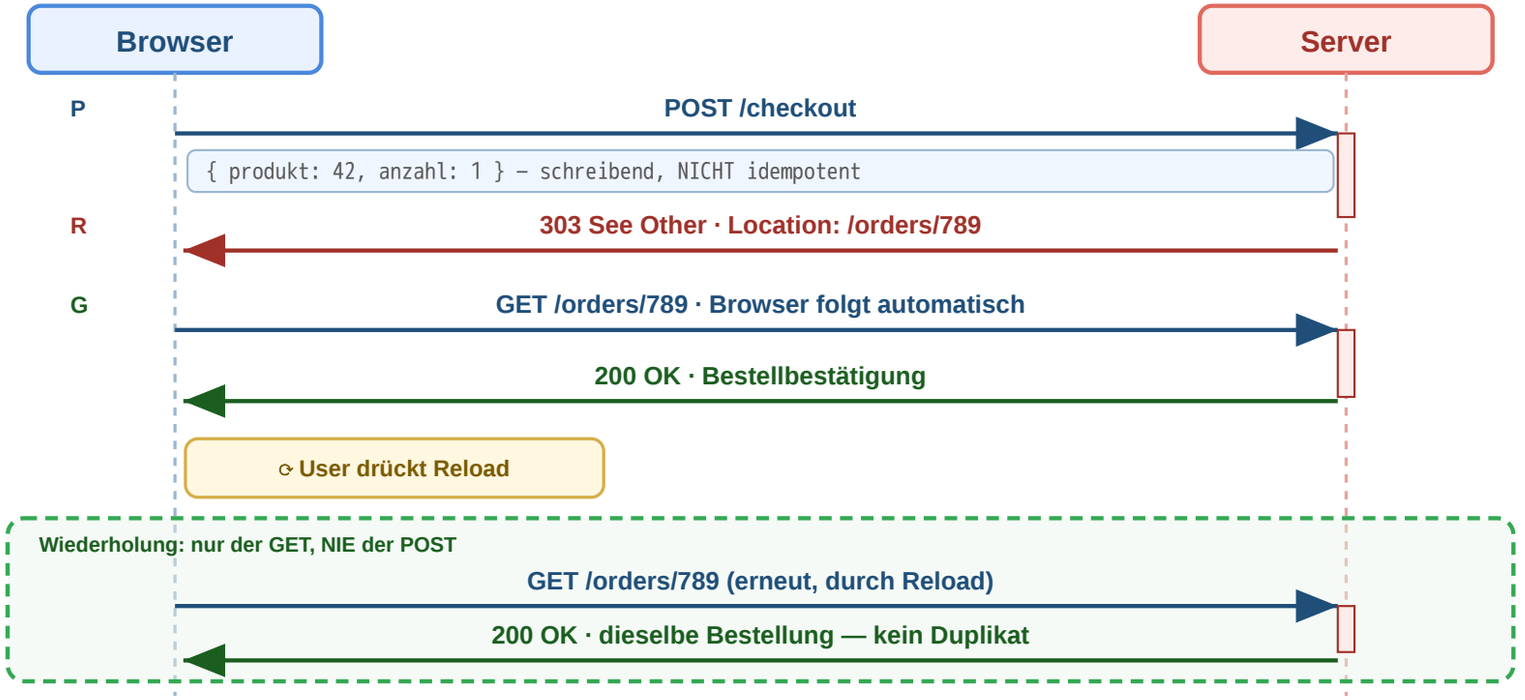
Ziel: Nach `POST /orders` zeigt der Browser die Bestätigungsseite. Drückt der Nutzer **Reload**, fragt der Browser: *"Formular erneut senden?"* — und sendet bei Bestätigung den POST **noch einmal**. Ergebnis: doppelte Bestellung. Auch *Zurück → Vor* oder das Speichern als Lesezeichen löst dieses Verhalten aus.

Wir wollen: den POST genau einmal ausführen und danach eine harmlose, reload-feste GET-Seite anzeigen.

Redirects und das Post/Redirect/Get-(PRG)-Pattern

Ziel: Nach `POST /orders` zeigt der Browser die Bestätigungsseite. Drückt der Nutzer **Reload**, fragt der Browser: *"Formular erneut senden?"* — und sendet bei Bestätigung den POST **noch einmal**. Ergebnis: doppelte Bestellung. Auch *Zurück → Vor* oder das Speichern als Lesezeichen löst dieses Verhalten aus.

Wir wollen: den POST genau einmal ausführen und danach eine harmlose, reload-feste GET-Seite anzeigen.



Code	Semantik	Typischer Einsatz
301	Permanent verschoben	Domain-Umzug
302/307	Temporär woanders	A/B-Test, Wartung
303	Nach POST auf GET	**Post/Redirect/Get (PRG)**
308	Permanent, Methode bleibt	API-Migration

Post/Redirect/Get (PRG) verhindert doppelte Bestellungen: Nach POST antwortet der Server mit `303 See Other` → der Browser folgt per GET → Neuladen der Seite wiederholt nur den GET, nicht den POST.

Workflow: Redirects in der Praxis

	Methode darf wechseln *POST→GET in der Praxis historisch üblich*	**Methode & Body bleiben** *strikt nach RFC — wichtig für APIs*

Permanent
Caches und Crawler sollen URL ersetzen
	301 Moved Permanently — Form-URL dauerhaft umgezogen
	`→ POST /kontaktformular`
	`{ name, email, nachricht }`
	`← 301 Moved Permanently`
	`Location: /v2/kontakt`
	`→ GET /v2/kontakt ⚠ POST→GET!`
	`← 200 OK (Body verloren)`
	→ Browser wechselt POST→GET — Form-Daten weg!
	Bei reinem GET (z.B. HTTP→HTTPS) bleibt GET. Für POST-APIs: 308.
		308 Permanent Redirect — API-Migration v1 → v2
		`→ POST /api/v1/orders`
		`{ produkt: 42, anzahl: 1 }`
		`← 308 Permanent Redirect`
		`Location: /api/v2/orders`
		`→ POST /api/v2/orders { produkt: 42 }`
		`← 201 Created`
		→ Methode UND Body bleiben — POST bleibt POST.
		Clients sollten neue URL persistent übernehmen.

Temporär
URL nur jetzt anders, später wieder Original
	302 Found — POST bei abgelaufener Session
	`→ POST /api/order (Session weg)`
	`{ produkt: 42, anzahl: 1 }`
	`← 302 Found`
	`Location: /login`
	`→ GET /login ⚠ POST→GET!`
	`← 200 OK · Login-Formular`
	→ POST-Daten weg — Bestellung muss nach Login neu eingegeben werd
	Für temporäre Verlegung von Schreib-Endpoints daher 307.
		307 Temporary Redirect — Wartung · Failover
		`→ POST /api/orders`
		`{ produkt: 42 }`
		`← 307 Temporary Redirect`
		`Location: https://b.api.example/orders`
		`→ POST b.api.example/orders { produkt: 42 }`
		`← 201 Created`
		→ Methode & Body bleiben — Originaladresse merken.
		Ideal für Wartungsfenster, Failover, A/B auf POST.

Sonderfall 303 See Other — erzwingt nach POST einen GET zur Bestätigungsseite (Post/Redirect/Get).

„Dein POST war erfolgreich, hol die Antwort hier per GET" — macht Reload unschädlich (siehe vorige Folie).
Anders als 307/308 wechselt die Methode hier bewusst auf GET — auch wenn das Original ein POST war.

Ziel: Eine Ressource ist nicht (mehr) unter der angefragten URL erreichbar — etwa nach Domain-Umzug, URL-Bereinigung, A/B-Test, Sprach-/Geo-Weiterleitung oder als Antwort auf einen schreibenden POST. Der Client soll **automatisch zur richtigen URL geleitet** werden, ohne dass der Nutzer eingreifen muss; Suchmaschinen und Caches sollen die Verschiebung korrekt interpretieren.
Wir wollen: mit dem passenden Statuscode steuern, ob die Weiterleitung dauerhaft oder temporär ist und ob Methode und Body erhalten bleiben.

Lang laufende Tasks: `202 Accepted` und Polling

Was tut der Server, wenn die Bearbeitung Sekunden oder Minuten dauert (CSV-Export, Video-Transcoding, LLM-Batch-Job)? Die Verbindung offen halten ist teuer und unzuverlässig. Das Standard-Muster ist ein **asynchroner Job mit Status-Polling** — eine Verkettung der bekannten Redirect-Bausteine.

```
# 1) Client startet den Task
POST /api/exports HTTP/1.1
Content-Type: application/json

{ "format": "csv", "range": "2026-Q1" }

# 2) Server quittiert SOFORT, ohne zu warten
HTTP/1.1 202 Accepted
Location: /api/jobs/abc123
Retry-After: 5

# 3) Client pollt den Job-Status (alle ~5 s)
GET /api/jobs/abc123 HTTP/1.1

HTTP/1.1 200 OK
Content-Type: application/json
{ "status": "processing", "progress": 0.42 }

# ⋯ weitere Polls ⋯

# 4) Fertig → Server schickt 303 zum Ergebnis
HTTP/1.1 303 See Other
Location: /api/exports/abc123.csv

# 5) Client folgt automatisch (PRG-Mechanik)
GET /api/exports/abc123.csv HTTP/1.1

HTTP/1.1 200 OK
```

Baustein	Funktion
`202 Accepted`	„Auftrag angenommen, Bearbeitung läuft"
`Location`	URL der **Job-Ressource** (nicht des Ergebnisses!)
`Retry-After`	Empfohlenes Poll-Intervall in Sekunden
Job: `200` + `status`	Zwischenstand: `queued`/`processing`/`done`/`failed`
Job: `303 See Other`	Endzustand → `Location` zeigt auf das fertige Artefakt

Wann nutzt man das?

- Server-seitig teure Berechnungen (Reports, ML-Inference, Renderings)
- Asynchrone Pipelines, die ohnehin in einem Worker laufen
- LLM-Batch-APIs (Anthropic, OpenAI) — gleiche Mechanik

Verwandte Muster

- *Long Polling*: Server hält die Anfrage offen, bis sich etwas ändert
- *WebSocket / SSE*: persistente Verbindung, Server pusht Events
- *Webhook*: Client gibt Callback-URL an, Server ruft zurück

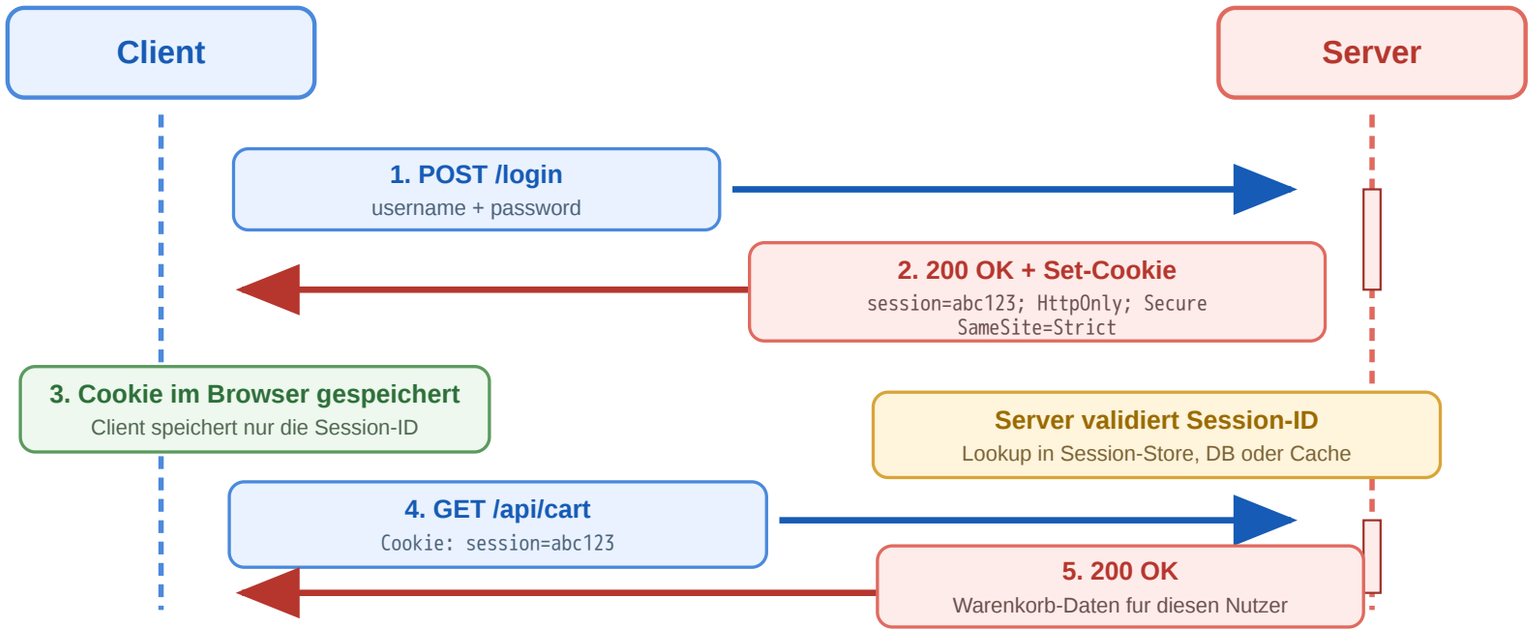
Faustregel: Sobald die Bearbeitung länger als ein Browser-Timeout (≈ 30 s) dauern kann, in Job-Ressource auslagern und per Polling beobachten.

HTTP Sessions and CORS

Leitfrage: Wie ermöglicht ein zustandsloses Protokoll persistente Sitzungen — und wie kontrolliert der Browser Cross-Origin-Zugriffe zwischen verschiedenen Domains?

Sessions und Cookies

HTTP ist zustandslos — aber der Online-Shop braucht Login und Warenkorb. **Cookies** lösen diesen Widerspruch:



Ablauf

1. `POST /login` mit Credentials → Server prüft.
2. Server antwortet mit `Set-Cookie: session=abc123` und legt die Session (Warenkorb, User-ID, …) serverseitig ab.
3. Browser speichert das Cookie für die Domain und sendet bei jedem weiteren Request automatisch `Cookie: session=abc123` mit.
4. Server liest die ID, schlägt die Session nach und kennt damit Identität und Zustand des Clients.

> Das Cookie enthält meist nur eine **opake ID** — der eigentliche Zustand liegt im Server (DB oder Redis).

Fundament: Same-Origin Policy — eine *Origin* ist die Kombination aus **Schema + Host + Port** (z.B. `https://shop.example:443`). Cookies gehören zu genau einer Origin:

- Der Browser sendet ein Cookie **nur zurück an die Domain, die es gesetzt hat** — nie an andere Domains.
- JavaScript auf `evil.example` kann das Cookie von `shop.example` **nicht lesen** (`document.cookie` ist origin-isoliert).
- Aber: bindet `news.example` ein Bild von `tracker.ad.net` ein, sendet der Browser dabei das Cookie von `tracker.ad.net` mit — ohne dass `news.example` es jemals sieht. Genau diese Regel ermöglicht Third-Party-Tracking (s. spätere Folien) und ist gleichzeitig die Basis, auf der die Angriffe der nächsten Folie (XSS, CSRF) ansetzen.

Angriffsvektoren auf Session-Cookies — und Schutz

Das Cookie ist die Identität. Wer die Session-ID besitzt, ist für den Server der Nutzer — kein Passwort nötig. Drei Wege, an die ID zu kommen, drei Schutzmechanismen am Cookie-Attribut.

Drei Angriffe

Angriff	Wie?	Schutz-Attribut
XSS (Cross-Site Scripting)	Eingeschleustes JS liest `document.cookie` und exfiltriert die Session-ID an einen fremden Server.	`HttpOnly` — Cookie ist für JS unsichtbar
CSRF (Cross-Site Request Forgery)	Bösartige Site löst im Hintergrund einen Request an die Bank/Shop-API aus; Browser sendet Cookie automatisch mit.	`SameSite=Strict` / `Lax` — Cookie wird bei Cross-Site-Requests nicht gesendet
MITM / Sniffing	Cookie wird im Klartext über HTTP übertragen; im offenen WLAN abgreifbar.	`Secure` — Cookie wird nur über HTTPS gesendet

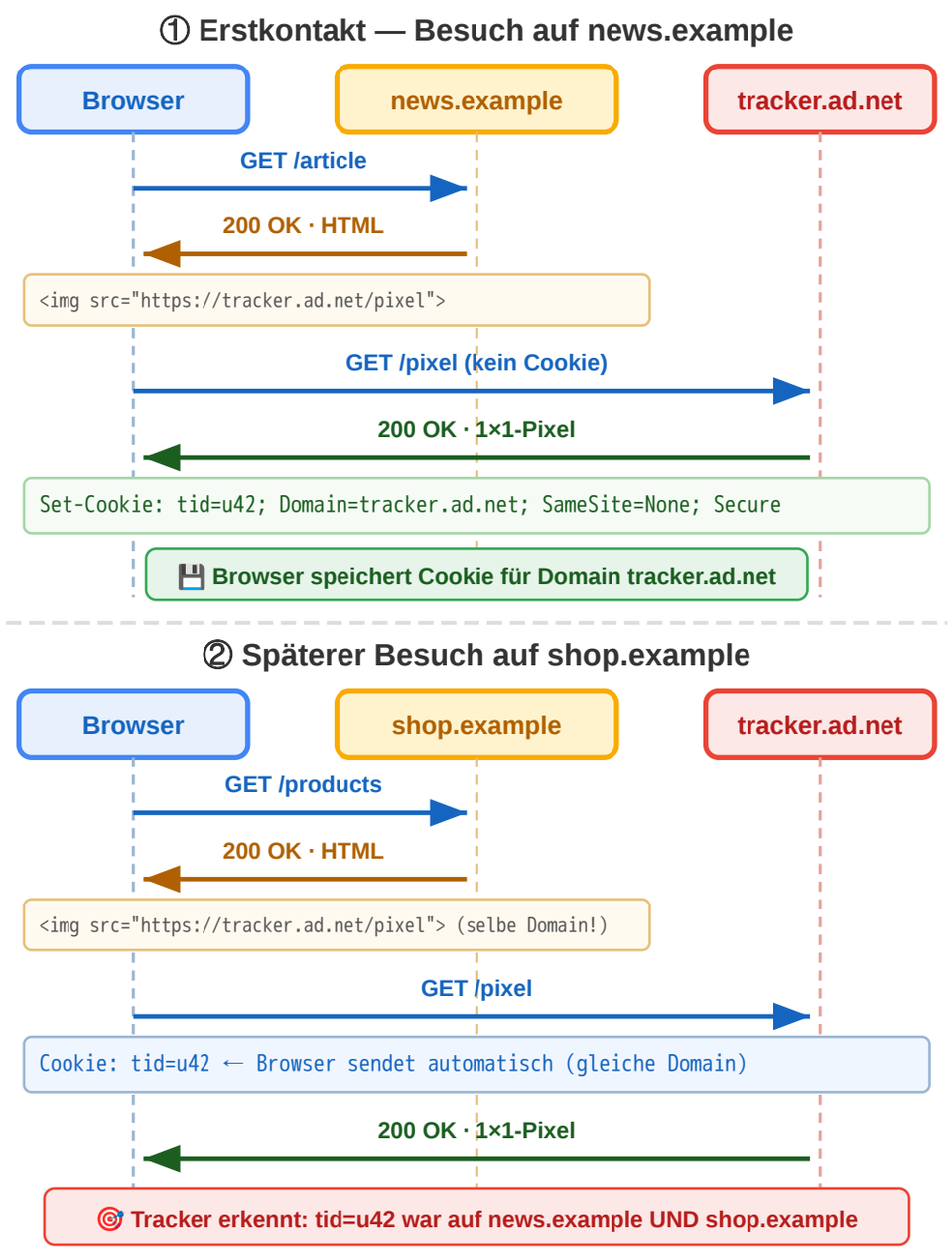
Sichere Konfiguration

Konfiguration	XSS	CSRF	Sniffing
`session=abc123`	✗	✗	✗
`…; HttpOnly`	✓	✗	✗
`…; HttpOnly; Secure`	✓	✗	✓
`…; HttpOnly; Secure; SameSite=Strict`	✓	✓	✓

```
Set-Cookie: session=abc123;
  HttpOnly; Secure; SameSite=Strict;
  Path=/; Max-Age=3600
```

Zusätzlich: Session-ID nach Login **rotieren** (gegen Session-Fixation) · Session serverseitig **invalidieren** bei Logout · kurze **Max-Age** + Sliding-Renewal.

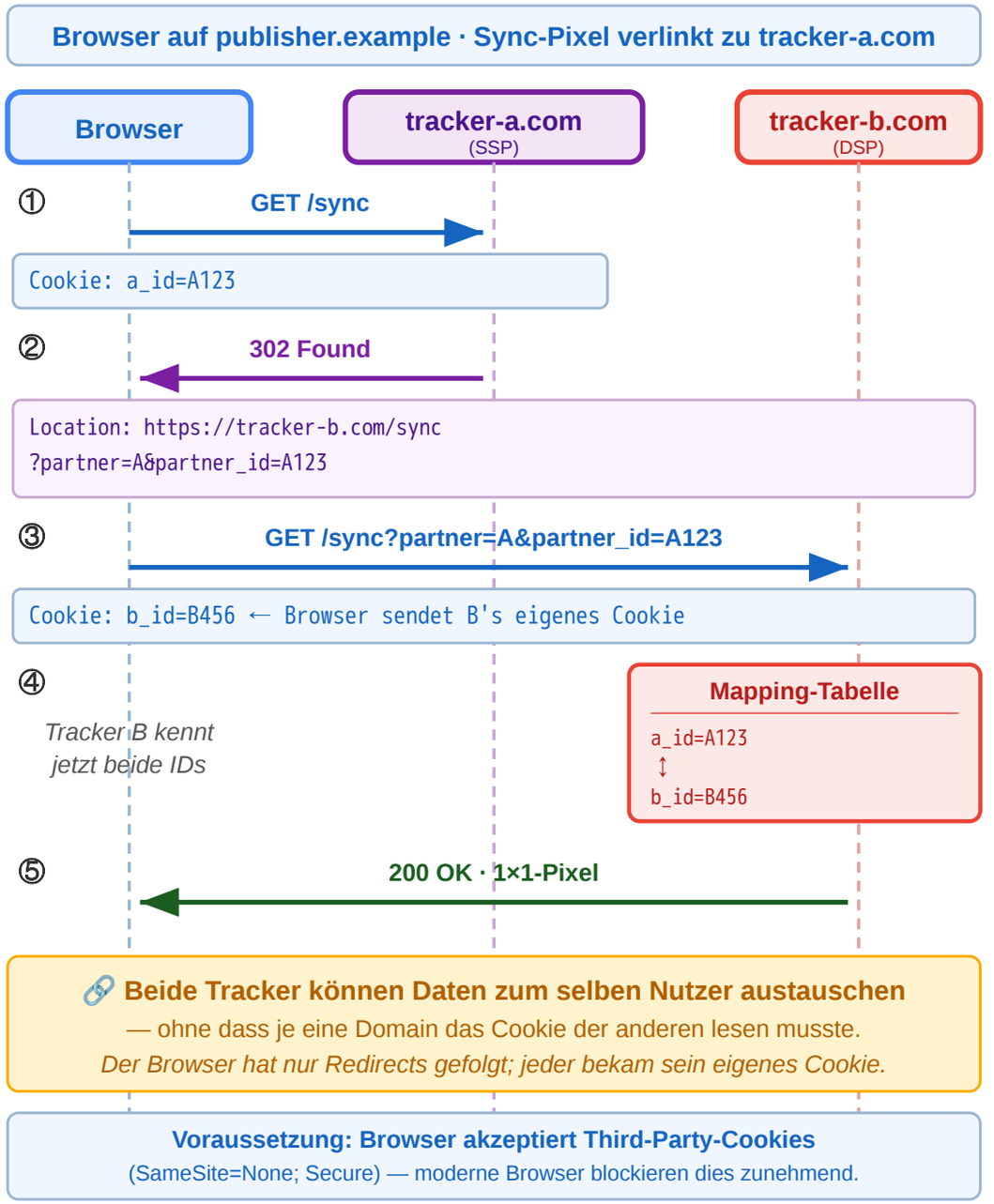
Cookie-Tracking I — Third-Party-Cookies



1. Mehrere Websites binden eine Ressource derselben Tracking-Domain ein (Bild, Skript, Werbe-Pixel, Social-Plugin).
2. Beim ersten Kontakt setzt der Tracker ein Cookie für **seine eigene Domain** — z.B. `tracker.ad.net`.
3. Bei jedem weiteren Besuch einer Site, die dieselbe Tracker-Ressource einbindet, sendet der Browser das Cookie automatisch mit.
4. Der Tracker erkennt: derselbe Browser war auf news.example **und** shop.example.

> Same-Origin bleibt gewahrt: Jede Domain liest nur ihre eigenen Cookies. Tracking funktioniert dadurch, dass viele Sites **dieselbe** Tracker-Domain einbinden — nicht weil Cookies geteilt werden.

Cookie-Tracking II — ID-Sync über Redirects



Akteure: Im Online-Werbemarkt vermitteln zwei Arten von Plattformen zwischen Publisher und Werbetreibendem:

- **SSP (Supply-Side Platform)** — auf der Verkäuferseite. Vermarktet die Werbeflächen eines Publishers (z.B. einer Nachrichtensite) und versteigert sie in Echtzeit an den höchstbietenden Käufer.
- **DSP (Demand-Side Platform)** — auf der Käuferseite. Kauft im Auftrag von Werbetreibenden gezielt Werbeflächen, um deren Zielgruppen anzusprechen.

Beide haben jeweils eigene Nutzer-IDs in eigenen Cookies — kennen aber die ID des Partners nicht. Same-Origin verbietet das direkte Lesen.

Lösung — Cookie-Syncing:

1. Sync-Pixel der SSP wird geladen; SSP liest ihr Cookie (`a_id=A123`).
2. SSP antwortet mit `302 Redirect` zur DSP — die eigene ID steckt **im URL-Parameter** (`?partner_id=A123`).
3. Browser folgt zur DSP und sendet dabei **deren eigenes Cookie** (`b_id=B456`) mit; in der URL liegt A's ID.
4. DSP speichert die Zuordnung `A123 ↔ B456` in ihrer Mapping-Tabelle und kann den Nutzer fortan beim Bidding wiedererkennen.

CORS — Same-Origin kontrolliert lockern

Warum es Same-Origin überhaupt gibt: Der Browser sendet Cookies *automatisch* an die Domain, die sie gesetzt hat. Ohne weitere Schutzregel könnte eine beliebige bösartige Site `evil.example` einfach `fetch("https://meinebank.example/konto")` aufrufen. Same-Origin Policy verhindert das: Skripte einer Origin dürfen Responses anderer Origins **nicht lesen**. Das ist die Sicherheitsbasis, auf der Cookies, Logins und sensible APIs im Browser überhaupt funktionieren können.

Problem: Genau dieselbe Regel blockiert auch legitime Anwendungsfälle. Eine SPA auf `https://app.example` will per `fetch()` mit `https://api.example` sprechen — beide gehören demselben Anbieter, sind aber unterschiedliche Origins. Ohne Ausnahme würde der Browser jeden Request blockieren.

CORS (*Cross-Origin Resource Sharing*) ist die kontrollierte Lockerung: Der **Server** entscheidet per Response-Header, welche fremden Origins lesend zugreifen dürfen. Der Browser erzwingt die Regel — das Skript bekommt das Response-Objekt nur dann zu sehen, wenn der Server zustimmt.

Zwei Arten von Cross-Origin-Requests:

Typ	Auslöser	Verhalten
Simple	`GET`/`HEAD`/`POST` mit nur Standard-Headern und Standard-Content-Types (`text/plain`, `multipart/form-data`, `application/x-www-form-urlencoded`)	Browser schickt den Request direkt; bei fehlendem `Access-Control-Allow-Origin` *verwirft* er die Antwort vor dem Skript.
Preflight-pflichtig	`PUT`/`DELETE`/`PATCH`, `Content-Type: application/json`, eigene Header wie `Authorization`	Browser schickt erst einen `OPTIONS`-Preflight (siehe nächste Folie) und nur bei

Server-Antwort (vereinfacht):

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://app.example
Content-Type: application/json

{ "user": "anna", ... }
```

Ohne `Access-Control-Allow-Origin` blockiert der Browser den Zugriff aus JavaScript (sichtbar als CORS-Fehler in der Konsole) — der Server kann dabei trotzdem schon einen Treffer protokolliert haben, weil der Request selbst ankam. CORS schützt also den *Client* vor unbeabsichtigtem Datenabfluss an fremde Origins, nicht den Server vor Anfragen.

Wichtig: CORS gilt nur für **Browser-JS** (`fetch`/`XHR`). Server-zu-Server-Requests, `curl`, oder Native-Apps sind nicht betroffen — die haben keine Same-Origin Policy.

CORS: Preflight mit `OPTIONS`

Ziel: Bevor der Browser einen Request schickt, der *mehr kann als ein klassisches HTML-Formular* (`PUT`/`DELETE`, JSON-Body, eigene Header wie `Authorization`), fragt er den Server erst per `OPTIONS`, ob die Operation überhaupt erlaubt ist. **Warum:** Simple Requests waren schon vor CORS möglich (HTML-Formular kann jede URL anfragen) — die kann der Browser nicht mehr zurückhalten. Alles darüber hinaus ist *neue* Fähigkeit, die XHR/`fetch` erst geschaffen hat — der Server muss sie deshalb explizit freigeben, bevor der Browser sie an eine fremde Origin schickt. So vermeidet man, dass eine bösartige Site neue Cross-Origin-Operationen am Server auslöst.

① Browser fragt nach:

```
OPTIONS /v1/orders HTTP/1.1
Host: api.example
Origin: https://app.example
Access-Control-Request-Method: PUT
Access-Control-Request-Headers:
  Authorization, Content-Type
```

② Server antwortet:

```
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: https://app.example
Access-Control-Allow-Methods: PUT, DELETE
Access-Control-Allow-Headers:
  Authorization, Content-Type
Access-Control-Max-Age: 86400
```

③ Nur bei passender Antwort schickt der Browser anschließend den eigentlichen `PUT /v1/orders` los — sonst wird er ohne Netzwerkversuch direkt im JS abgebrochen.

Was der Browser konkret prüft:

Antwort-Header	Prüfung
`Allow-Origin`	matched die aufrufende Origin?
`Allow-Methods`	enthält die geplante Methode (`PUT`)?
`Allow-Headers`	enthält alle nicht-Standard-Header (`Authorization`, eigene)?

`Max-Age: 86400` — der Browser cacht die Preflight-Antwort 24 h und spart sich `OPTIONS` für Folgeaufrufe. Policy-Änderungen greifen erst nach Cache-Ablauf.

CORS und Credentials

> **Default:** Cross-Origin-`fetch` sendet **keine Cookies** und keine `Authorization`-Header — auch wenn der Browser sie für die Zieldomain gespeichert hätte. Das schützt vor unbeabsichtigtem Mitsenden von Anmeldedaten an fremde Origins.

Damit Cookies/Auth mitgehen — drei Bedingungen müssen alle erfüllt sein:

1. Client signalisiert Absicht:

```
fetch("https://api.example/me", {
  credentials: "include"   // statt default "same-origin"
})
```

2. Server erlaubt explizit:

```
Access-Control-Allow-Origin: https://app.example
Access-Control-Allow-Credentials: true
```

> **Wildcard verboten:** `Access-Control-Allow-Origin: *` ist mit `Allow-Credentials: true` **nicht** kombinierbar. Die Origin muss explizit genannt werden, sonst lehnt der Browser ab.

3. Cookie selbst muss cross-site-fähig sein:

```
Set-Cookie: session=abc123;
  HttpOnly; Secure;
  SameSite=None     ← Pflicht für Cross-Origin
```

Das Zusammenspiel mit `SameSite`:

`SameSite`	**Verhalten bei Cross-Origin**
`Strict`	Cookie wird **nie** an fremde Origin gesendet — auch bei expliziter CORS-Freigabe nicht
`Lax` (default)	Cookie geht nur bei *Top-Level-Navigation* mit, nicht bei `fetch`/XHR
`None` + `Secure`	Cookie wird bei Cross-Origin gesendet, sofern CORS zustimmt

Typisches Setup für SPA + getrennte API-Domain:

- Frontend `app.example`, API `api.example`
- Session-Cookie: `SameSite=None; Secure; HttpOnly`
- CORS auf API: `Allow-Origin: https://app.example` + `Allow-Credentials: true`
- Frontend-`fetch` mit `credentials: "include"`

Alternativ — wenn man `SameSite=Strict` halten will: Frontend und API auf derselben Site (z.B. `app.example` + `app.example/api/*`) hosten oder ein BFF-Pattern verwenden.

Warum so streng? Ohne diese Einschränkungen könnte eine bösartige Seite per `fetch` Login-Cookies des Nutzers an authentifizierte APIs mitschicken — das alte CSRF-Problem in CORS-Form. Die Dreifach-Zustimmung (Client + Server + Cookie) macht das praktisch unmöglich.

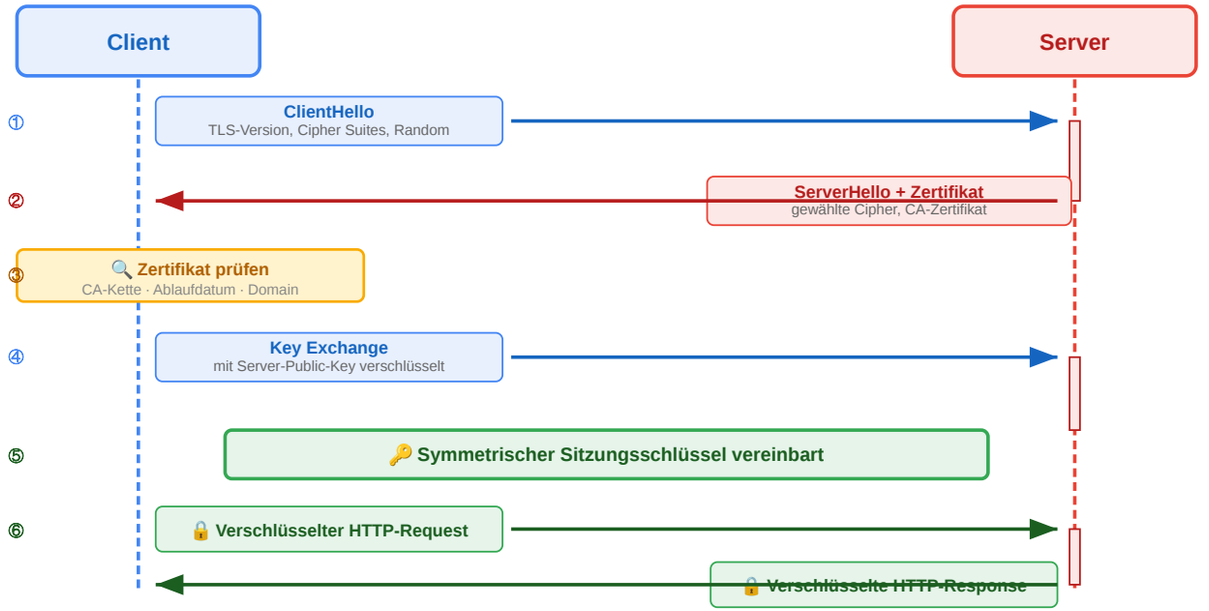
HTTP Authentifizierung und Sicherheit

Leitfrage: Wie schützt HTTP Identität, Vertraulichkeit und Integrität auf der Leitung — und welche Mechanismen autorisieren API-Zugriff in unterschiedlichen Szenarien?

HTTPS und TLS

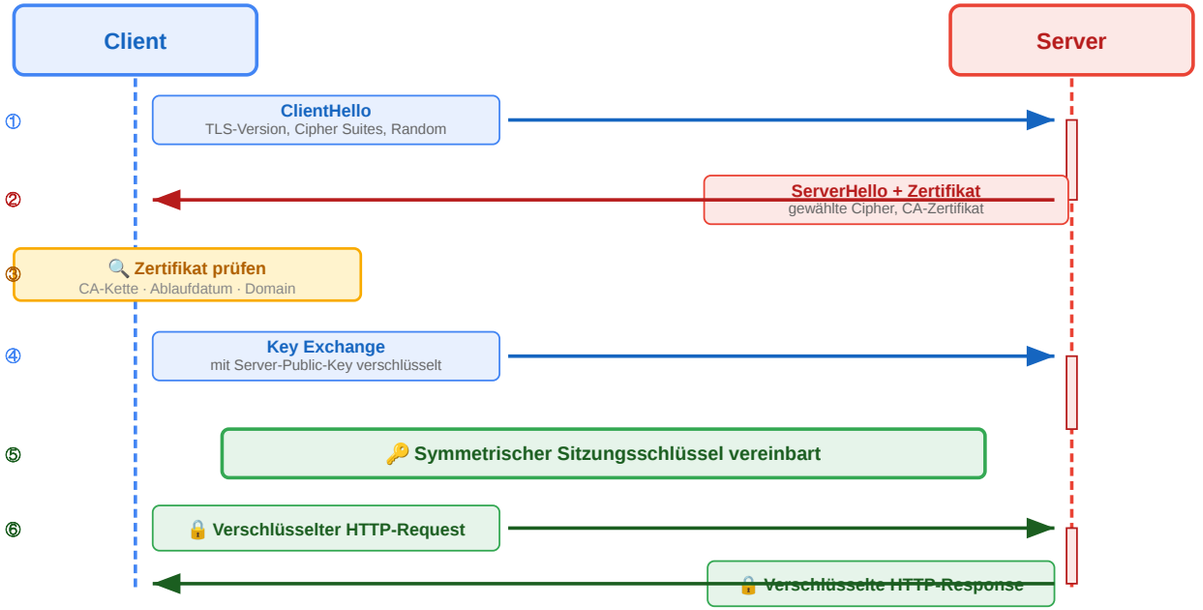
Eigenschaft	HTTP	HTTPS
Vertraulichkeit	Klartext lesbar	Verschlüsselt
Integrität	Manipulierbar	Manipulation erkannt
Authentizität	Keine Garantie	Zertifikat bestätigt

Diskussionsfrage: Ein Angreifer im selben WLAN liest den Netzwerkverkehr mit. Wie schützt TLS hier und was schützt es nicht?



HTTPS und TLS

Eigenschaft	HTTP	HTTPS
Vertraulichkeit	Klartext lesbar	Verschlüsselt
Integrität	Manipulierbar	Manipulation erkannt
Authentizität	Keine Garantie	Zertifikat bestätigt



Diskussionsfrage: Ein Angreifer im selben WLAN liest den Netzwerkverkehr mit. Wie schützt TLS hier und was schützt es nicht?

Lösung:

- **Vertraulichkeit** schützt — das WLAN-Sniffing sieht nur verschlüsselten Datenverkehr.
- **Integrität** schützt — manipulierte Pakete würden auffallen.
- **Authentizität** schützt im engeren Sinne nicht vor *passivem Mitlesen*, ist aber Voraussetzung dafür, dass der Schutz überhaupt greift: ohne Zertifikatsprüfung könnte ein aktiver Angreifer einen eigenen TLS-Endpunkt vorspielen (MITM) und die Verbindung im Klartext mitlesen.

Was TLS *nicht* schützt: DNS-Anfragen und Traffic-Muster sind weiter sichtbar — der Angreifer sieht *welche* Site besucht wird, nur nicht *was* übertragen wird.

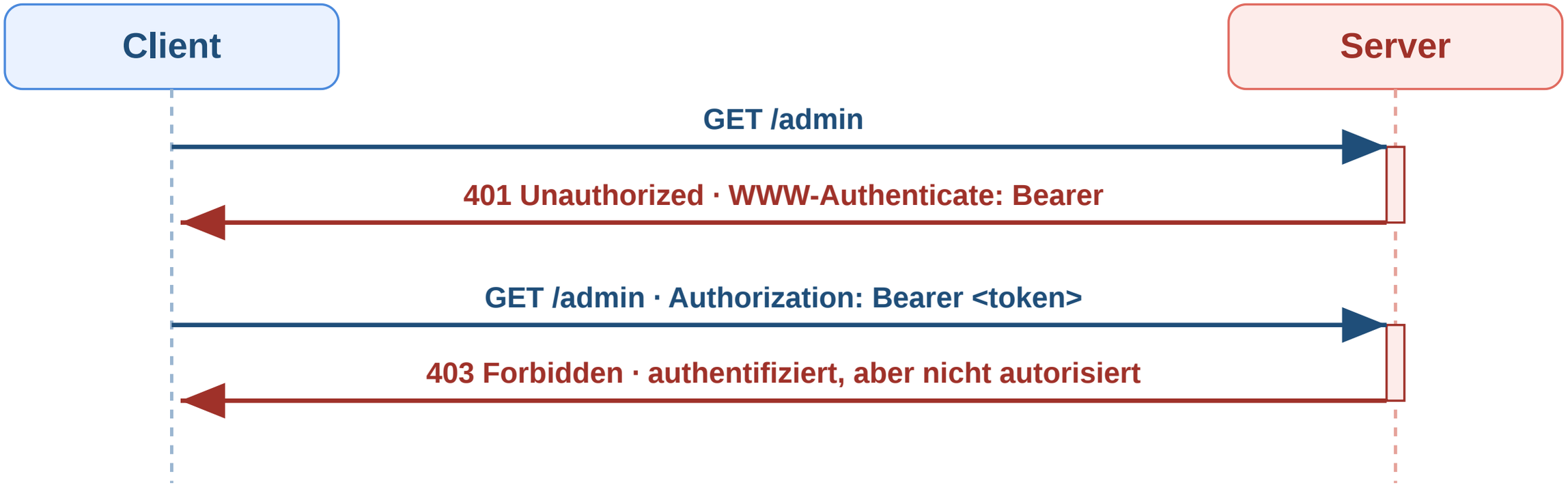
Authentifizierung vs. Autorisierung — Überblick

> **Authentifizierung (AuthN):** *Wer bist du?* — Identitätsnachweis.
> **Autorisierung (AuthZ):** *Was darfst du?* — Berechtigung für eine konkrete Operation oder Ressource.

Mechanismus	Funktionsweise	State	Typischer Einsatz
API Key	Statischer Schlüssel im Header	Server-Lookup	M2M-APIs, CLI-Tools, Webhooks
Session Cookie	Opake Session-ID im Cookie + Server-Store	stateful	Klassische Web-Apps
Bearer Token (JWT)	Signierter Token mit Claims	stateless	SPAs, Mobile, Microservices
OAuth 2.0	Delegierter Token vom Authorization Server	je nach Flow	„Login mit Google", Third-Party-Integrationen

Die vier Mechanismen sind keine Alternativen für dasselbe Problem, sondern adressieren unterschiedliche Szenarien. Auf den folgenden Folien jeweils einer im Detail.

Workflow: Authentifizierung mit 401 und 403



Status	Bedeutung
`401 Unauthorized`	Keine gültigen Anmeldedaten vorhanden
`403 Forbidden`	Identität bekannt, aber Zugriff nicht erlaubt

Mechanismus 1 — API Key

```
GET /v1/charges HTTP/1.1
Host: api.stripe.com
Authorization: Bearer sk_live_4eC39HqLyjWDarjtT1zdp7dc
```

Alternativ über eigenen Header:

```
GET /v1/charges HTTP/1.1
X-API-Key: sk_live_4eC39HqLyjWDarjtT1zdp7dc
```

Funktionsweise:

1. Provider stellt den Schlüssel einmal aus; Konsument speichert ihn sicher (Env-Variable, Secret-Manager).
2. Bei jedem Request schickt der Client den Schlüssel im Header mit.
3. Server schlägt den Schlüssel in einer DB nach, ermittelt das zugehörige Account und prüft Quotas und Scopes.

✓ Vorteil	✗ Nachteil
Trivial zu implementieren	Schlüssel = Vollzugriff
Funktioniert ohne Browser	Kein automatischer Ablauf
Cachebar, loggbar	Selten an Nutzer gebunden

Praxis: Stripe, GitHub PATs, OpenAI-API, Slack-Bots — überall dort, wo *Server* mit *Server* spricht und kein interaktiver Nutzer dahinter steht.

Schlüssel nie ins Repo, nie in URLs, nie in Logs. Bei Leak: sofort rotieren.

Mechanismus 2 — Session Cookie

```
# 1) Login
POST /login HTTP/1.1
{ "username": "anna", "password": "..." }

# 2) Server-Response
HTTP/1.1 200 OK
Set-Cookie: session=abc123;
  HttpOnly; Secure; SameSite=Strict; Path=/

# 3) Folge-Request – Browser sendet automatisch
GET /dashboard HTTP/1.1
Cookie: session=abc123
```

Funktionsweise:

1. Login: Server prüft Credentials, legt Session-Eintrag in DB/Redis ab, schickt Session-ID im Cookie.
2. Browser speichert Cookie domain-gebunden und sendet es bei jedem weiteren Request mit.
3. Server schlägt ID im Session-Store nach → kennt Identität und Zustand.

✓ Vorteil	✗ Nachteil
Browser handhabt Cookie automatisch	Server muss Session-Store skalieren
Logout = sofortige serverseitige Invalidierung	Stateful — Sticky Sessions oder geteilter Cache
Mit `HttpOnly`/`Secure`/`SameSite` robust	Schlecht für Cross-Domain-APIs

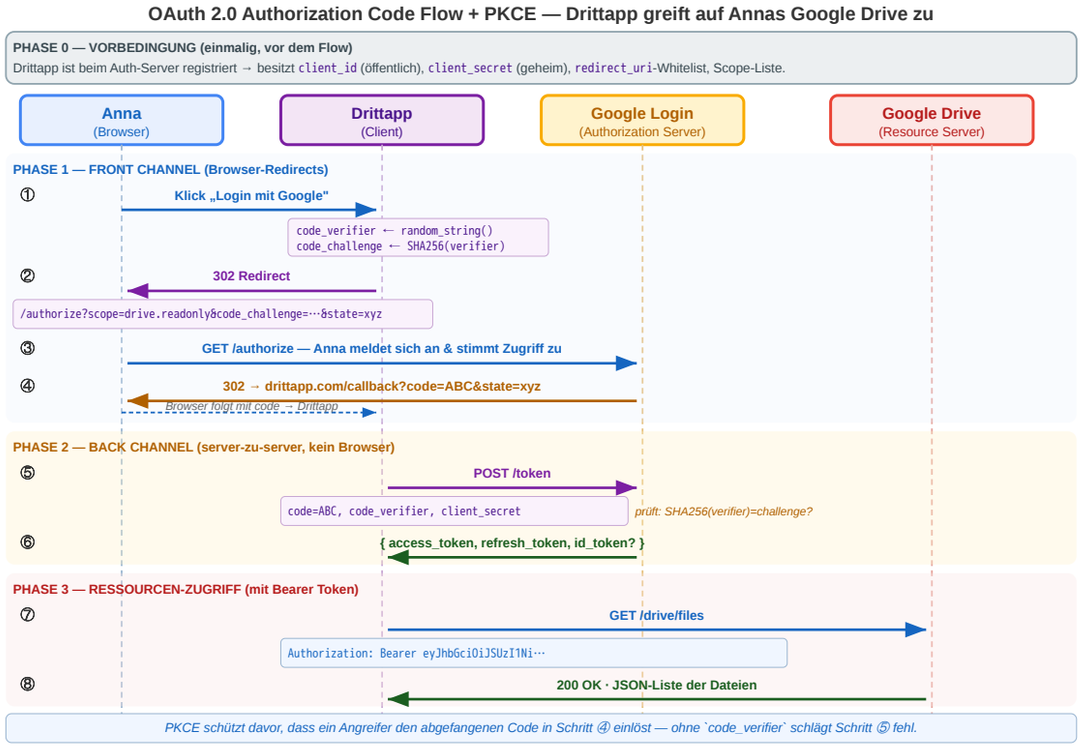
> **Praxis:** Server-gerenderte Apps (Rails, Django, Laravel, Spring MVC) und moderne **BFF**-Architekturen ("Backend for Frontend"), bei denen das SPA-Frontend über einen eigenen Backend-Proxy gegen die API spricht.

Mechanismus 3 — OAuth 2.0 (Überblick)

Problem: Eine App will im Auftrag des Nutzers auf eine *fremde* Ressource zugreifen (Kalender, Mails, Drive) — der Nutzer soll *nicht* sein Passwort weitergeben.

Vier Akteure:

Akteur	Rolle	Beispiel
Resource Owner	Der Nutzer	Anna
Client	App, die zugreifen will	Drittapp
Authorization Server	Vergibt Tokens	Google Login
Resource Server	Hält die Daten	Google Drive API



> **Wichtig:** OAuth 2.0 ist *Autorisierung* (Delegation), nicht primär *Authentifizierung*. Für „Login mit Google" wird **OIDC** verwendet, das auf OAuth aufsetzt und einen JWT als ID-Token zurückgibt. Details sind eigenes Thema.

Mechanismus 4 — Bearer Token (JWT)

Warum brauchen wir JWTs? OAuth (vorige Folie) liefert dem Client einen *Access Token* — aber was steckt eigentlich drin? In verteilten Architekturen (Microservices, Edge-Deployments, Multi-Region) muss jeder Service einen Aufrufer authentifizieren können, **ohne mit einer zentralen Session-DB zu sprechen**. JWTs lösen genau das: ein vom Auth-Server signiertes, selbstbeschreibendes Token, das alle Identitätsdaten selbst trägt — *Server-zu-Server-Authentifizierung* wird zur reinen lokalen Signaturprüfung, ohne Roundtrip zum Auth-Server.

JWT = *JSON Web Token* (RFC 7519). Drei Base64-URL-codierte Teile, durch Punkte getrennt — `Header.Payload.Signature`.

Teil	Inhalt
Header	Algorithmus (`alg`) und Token-Typ (`typ`)
Payload	*Claims*: `sub` (User-ID), `exp` (Ablauf), `iat`, `iss`, Rollen, Tenant — beliebige JSON-Felder
Signature	Kryptografische Signatur über `Header.Payload` mit Server-Secret (HMAC) oder Private Key (RSA/ECDSA)

Warum drei Teile, warum diese Codierung?

- **Base64-URL** statt Plain-JSON: macht den Token URL-/Header-sicher.
- **Signatur trennbar**: Server prüft nur den hinteren Teil; *was* signiert wurde, bleibt für Debugging lesbar.
- **Header trägt** `alg`: Der Verifier weiß *bevor* er den Inhalt parst, mit welchem Verfahren er prüfen muss.

Wie funktioniert die Verifikation — und warum reicht das?

1. Server berechnet aus `Header.Payload` + bekanntem Secret/Public-Key erneut die Signatur.
2. Vergleicht mit der mitgesendeten Signatur (timing-safe). Stimmt sie nicht → 401.
3. Liest Claims aus der Payload. Prüft `exp` (nicht abgelaufen?), `iss`/`aud` (vom richtigen Aussteller, für mich bestimmt?).
4. **Kein DB-Lookup nötig** — alle Identitätsdaten stecken im Token selbst.

Bei asymmetrischer Signatur (RSA/ECDSA) holt der prüfende Service den Public Key des Auth-Servers einmalig über **JWKS** (JSON Web Key Set, meist `/.well-known/jwks.json`) und cacht ihn. Er muss kein Geheimnis hüten.

JWT — Beispiel, Nutzen und Trade-offs

Beispiel-Request:

```
GET /api/me HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.
  eyJzdWIiOiI0MiIsImV4cCI6MTcxNTAwMDAwMCwicm9sZSI6ImFkbWluIn0.
  Mz5e_kJ4qP···SIG
```

Decodiert (Base64-URL):

```
{"alg":"RS256","typ":"JWT"}                    // Header
{"sub":"42","exp":1715000000,"role":"admin"}   // Payload
```

Warum nützlich: horizontale Skalierung ohne Session-Store; weiterreichbar zwischen Microservices; asymmetrische Signatur → Public-Key-Verifikation überall, keine geteilten Geheimnisse.

Preis — Widerruf vor Ablauf ist schwer. Drei Muster:

Muster	Kerngedanke
Kurze `exp` + Refresh-Token	Access-Token 5–15 min; serverseitig invalidierbares Refresh-Token verlängert
Server-Blocklist	`jti` widerrufener Tokens in Redis prüfen — sofortige Wirkung, dafür wieder stateful
Key-Rotation	Signing Key tauschen — invalidiert *alle* Tokens, Notbremse

> **Faustregel:** Wer sofortigen, selektiven Widerruf braucht (Compliance, Finanzaufsicht, Healthcare), nimmt eher Session Cookies. JWTs sind für „eventually consistent" Identitätsaussagen gemacht, nicht für Echtzeit-Zugriffskontrolle.

Zusammenfassung — HTTP in der Praxis

Semantik statt Syntax

- **Methoden** kommunizieren Absicht — `GET` liest, `POST` schreibt nicht-idempotent, `PUT`/`DELETE` sind idempotent.
- **Statuscodes** sind die maschinenlesbare Antwort — `2xx` Erfolg, `3xx` Weiterleitung, `4xx` Client-Fehler, `5xx` Server-Fehler.
- **Header** steuern Format, Aushandlung, Caching, Auth und Sessions — kein Beiwerk.

Performance: Caching mit Trade-off

- `Cache-Control: max-age` reduziert Last und Latenz — Preis: zeitweise veraltete Daten.
- `ETag` + `If-None-Match` ermöglichen Validierung ohne erneute Übertragung (`304 Not Modified`).
- Cache-Ebenen wirken kumulativ: Browser, CDN/Proxy, Application-Cache.

Navigation und Schreiboperationen

- **PRG** (`POST` → `303` → `GET`) verhindert doppelte Verarbeitung bei Reload.
- **301/308** dauerhaft, **302/307** temporär; **307/308** erhalten Methode und Body — Pflicht für APIs.

Zustand trotz Zustandslosigkeit

- **Cookies** stabilisieren Sessions und werden domain-gebunden mitgesendet (Same-Origin).
- Sicherheits-Attribute orthogonal: `HttpOnly` (XSS), `SameSite` (CSRF), `Secure` (Sniffing).
- Dieselbe Mechanik ermöglicht Tracking — direkt über Third-Party-Cookies oder per ID-Sync via Redirect.

Transport: HTTPS / TLS

- Vertraulichkeit, Integrität, Authentizität — aber nur für die Leitung, nicht für die Anwendungslogik.

HTTP bleibt simpel auf Protokollebene, wird aber durch Header und Statuscodes zum ausdrucksstarken Steuerungsmechanismus. Caching ist der wichtigste Performance-Hebel — aber jede Caching-Entscheidung ist ein Trade-off zwischen Aktualität und Last. Redirects steuern Navigation, PRG verhindert Doppelverarbeitung. Cookies ermöglichen Zustand trotz Zustandslosigkeit — aber nur mit korrekter Konfiguration. TLS sichert den Transport.

REST als Architekturidee

Leitfrage: Was unterscheidet eine RESTful API von einem beliebigen HTTP-Endpunkt, der nur JSON ausliefert?

REST kurz eingeführt

> **REST** steht für **Representational State Transfer**.

REST entstand nicht als Framework oder Produkt, sondern als Beschreibung dafuer, **wie das Web bereits erfolgreich funktioniert**:

- Ressourcen haben stabile Adressen (`/products/42`)
- Clients navigieren über standardisierte HTTP-Operationen
- Server liefern **Repräsentationen** dieser Ressourcen
- Infrastruktur wie Browser, Caches und Proxies kann mitarbeiten

Entstehungskontext

- Ende der 1990er Jahre im Umfeld der Web-Architektur
- Geprägt durch **Roy Fielding**
- Ausgangsfrage: Warum skaliert das Web trotz seiner Einfachheit so gut?



Für die Praxis: REST ist zuerst ein **Programmier- und Entwurfsparadigma für verteilte Client-Server-Systeme**. Die formale Einordnung als Architekturstil kommt im naechsten Schritt.

REST: Ein Architekturstil, kein Protokoll

REST ist kein einzelnes Protokoll, sondern ein Satz von Constraints für verteilte Systeme.

Constraint	Bedeutung	Konsequenz
Client-Server	Klare Rollentrennung	Client und Server entwickeln sich unabhängig
Stateless	Jeder Request enthält alle nötigen Informationen	Server skaliert horizontal, kein Session-Zustand
Cacheable	Responses deklarieren ihre Cachebarkeit	Infrastruktur kann Caching transparent umsetzen
Uniform Interface	Einheitliche Schnittstelle für alle Ressourcen	Generische Werkzeuge (Browser, curl, Proxies) funktionieren
Layered System	Client weiß nicht, ob er direkt mit dem Server spricht	Load Balancer, CDN, API Gateway transparent einfügbar
Code on Demand (optional)	Server kann ausführbaren Code liefern	JavaScript im Browser

Kernidee

> REST nutzt die **vorhandene Web-Infrastruktur** (HTTP, URIs, Caching, Proxies) als Anwendungsplattform — statt ein eigenes RPC-Protokoll darüber zu bauen.

Welchen Constraint verletzt dieses Design?

Szenario 1: Der Online-Shop speichert den Warenkorb in einer serverseitigen Session. Beim nächsten Request muss der Client denselben Server treffen (Sticky Sessions).

Szenario 2: Die API liefert bei `GET /api/products` keinen `Cache-Control`-Header. Jeder Request geht bis zum Origin-Server.

Szenario 3: Der Client kennt die interne Datenbankstruktur und baut SQL-artige Queries: `GET /api/query?table=products&where=price>50`

Szenario	Verletzter Constraint	Konsequenz
1: Sticky Sessions	**Stateless**	Horizontale Skalierung eingeschränkt, Server-Ausfall = Session verloren
2: Kein Cache-Control	**Cacheable**	CDN nutzlos, unnötige Serverlast, höhere Latenz
3: SQL-artige Queries	**Uniform Interface** + **Layered System**	Client an Interna gekoppelt, keine Abstraktion möglich

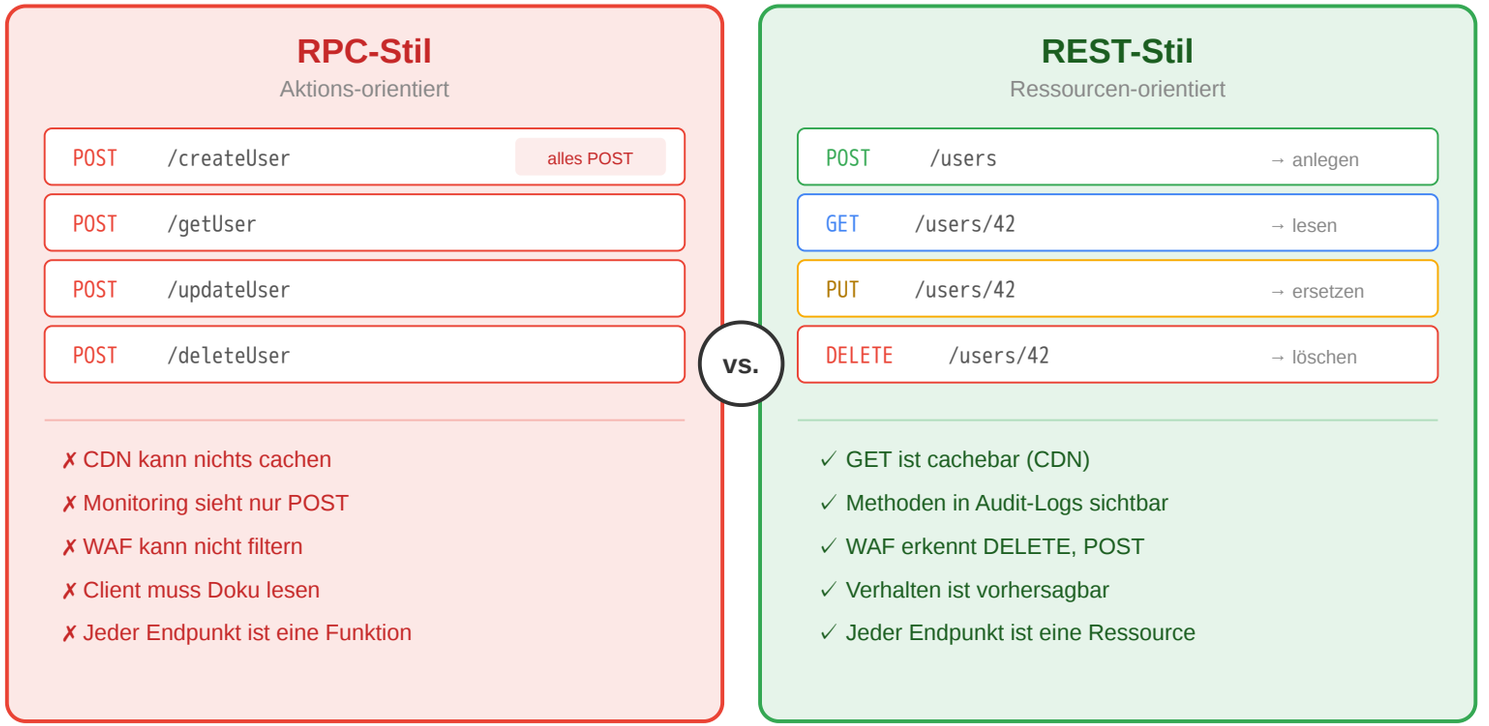
REST vs. RPC im Vergleich

RPC-Stil (aktionsorientiert)

- Jeder Endpunkt ist eine **Funktion**
- HTTP-Methode immer POST
- Infrastruktur kann nichts cachen
- Client muss Doku lesen

REST-Stil (ressourcenorientiert)

- Jeder Endpunkt ist eine **Ressource**
- HTTP-Methoden drücken Absicht aus
- GET-Requests sind cachebar
- Verhalten ist vorhersagbar



Warum das für Infrastruktur relevant ist

CDN, WAF und Monitoring verstehen HTTP-Semantik: GET wird gecacht, POST als schreibend erkannt, DELETE in Audit-Logs hervorgehoben. RPC-Stil verliert all das — jeder POST sieht gleich aus.

Ressourcen und Repräsentationen

Eine **Ressource** ist ein fachliches Konzept. Was der Client empfaengt, ist eine **Repräsentation**.

Konzept	Bedeutung
Ressource	Das fachliche Ding (existiert im System)
URI	Die Adresse der Ressource (identifiziert sie eindeutig)
Repräsentation	Die konkrete Darstellung (JSON, HTML, CSV - je nach `Accept`-Header)
Zustandsübergang	Aktion auf der Ressource (GET liest, POST erzeugt, DELETE entfernt)

> **Content Negotiation:** Client und Server einigen sich über `Accept` und `Content-Type` auf das Format.

> Dieselbe Ressource kann also über dieselbe URI identifiziert werden, während ihre konkrete Darstellung je nach Client unterschiedlich ausfaellt.

Wann stößt REST an Grenzen?

Situation	REST-Problem	Alternative
Client braucht Daten aus 5 Ressourcen gleichzeitig	5 separate Requests → Latenz	**GraphQL**: Ein Query, ein Response
Echtzeit-Updates (Chat, Live-Ticker)	Polling ist ineffizient	**WebSockets**: bidirektionale Verbindung
Interne Microservice-Kommunikation	JSON-Overhead, keine Typsicherheit	**gRPC**: binäres Protokoll, Protobuf-Schema
Komplexe Aktionen (Batch-Operationen)	Passt nicht in CRUD	**RPC-Endpunkte** als Ergänzung

Kein Entweder-Oder

Viele Systeme **kombinieren** Stile: REST für öffentliche APIs, GraphQL für flexible Frontends, gRPC für interne Services. Die Wahl hängt von **Nutzungsszenarien** ab — nicht von Dogma.

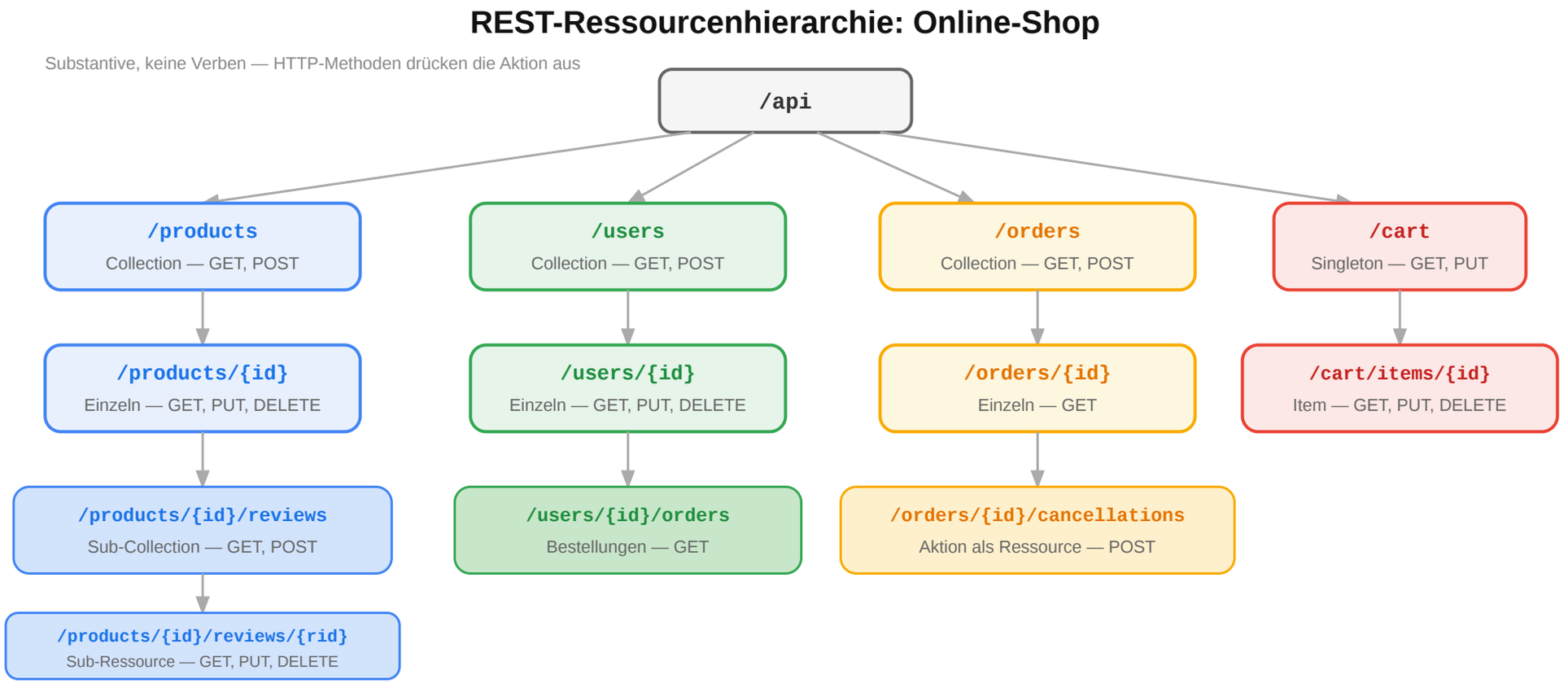
> REST ist kein JSON-over-HTTP. Es ist ein Architekturstil mit sechs Constraints, die das Web skalierbar, cachebar und erweiterbar machen. Die Stärke liegt in der konsequenten Nutzung bestehender Web-Infrastruktur. Wer die Constraints versteht, kann beurteilen, wann REST die richtige Wahl ist und wann ein anderer Stil besser passt.

Ressourcen und URI-Design

Leitfrage: Wie werden fachliche Objekte so als Ressourcen modelliert, dass eine API verständlich und stabil bleibt?

Ressourcenorientiertes Design: Online-Shop

Der erste Schritt beim API-Design ist die Identifikation der **Ressourcen** — die fachlichen Dinge, mit denen Clients interagieren:



URI-Design-Regeln

Regel	Gut	Schlecht	Warum
Plural für Collections	`/products`	`/product`	Collection enthält viele
Kleinschreibung, Bindestriche	`/order-items`	`/OrderItems`	URL-Konvention
Keine Verben im Pfad	`POST /orders`	`POST /createOrder`	Methode drückt Aktion aus
Hierarchie = Zugehörigkeit	`/users/42/orders`	`/getUserOrders?userId=42`	Beziehung sichtbar
Query für Filter/Sortierung	`/products?sort=price`	`/products/sortByPrice`	Pfad = Identität, Query = Parameter

Anti-Patterns erkennen

```
POST /api/getUser             →  ?
POST /api/deleteAccount       →  ?
GET  /api/search?action=find  →  ?
POST /api/doPayment           →  ?
```

Ressourcenorientierte Fassung

```
POST /api/getUser             →  GET    /users/{id}
POST /api/deleteAccount       →  DELETE /accounts/{id}
GET  /api/search?action=find  →  GET    /products?q=...
POST /api/doPayment           →  POST   /orders/{id}/payments
```

Übung: API-Design für eine Domäne

Aufgabe: Entwerfen Sie die REST-Ressourcen für ein **Universitäts-Kursverwaltungssystem**. Welche Ressourcen gibt es? Welche URIs? Welche Hierarchien?

Hinweis: Denken Sie an Kurse, Vorlesungen, Studierende, Einschreibungen, Noten.

Übung: API-Design für eine Domäne

Aufgabe: Entwerfen Sie die REST-Ressourcen für ein **Universitäts-Kursverwaltungssystem**. Welche Ressourcen gibt es? Welche URIs? Welche Hierarchien?

Hinweis: Denken Sie an Kurse, Vorlesungen, Studierende, Einschreibungen, Noten.

Eine mögliche Lösung

Fachliches Konzept	URI	Methoden
Alle Kurse	`/courses`	GET, POST
Ein Kurs	`/courses/web-eng`	GET, PUT, DELETE
Vorlesungen eines Kurses	`/courses/web-eng/lectures`	GET, POST
Einschreibungen	`/courses/web-eng/enrollments`	GET, POST

Fachliches Konzept	URI	Methoden
Eine Einschreibung	`/courses/web-eng/enrollments/42`	GET, PUT
Noten eines Studierenden	`/students/42/grades`	GET
Note in einem Kurs	`/students/42/grades/web-eng`	GET, PUT

> **Diskussion:** Sollte die Note unter `/students/42/grades/web-eng` oder unter `/courses/web-eng/grades/42` liegen? Beide sind valide — die Wahl hängt davon ab, **wer der primäre Nutzer der API** ist.

CRUD-Mapping auf HTTP

Operation	HTTP	URI	Body	Response
Liste lesen	`GET`	`/products`	—	`200` + Array
Einzeln lesen	`GET`	`/products/42`	—	`200` + Objekt
Anlegen	`POST`	`/products`	Neues Objekt	`201` + Location
Vollständig ersetzen	`PUT`	`/products/42`	Vollständig	`200` / `204`
Teilweise ändern	`PATCH`	`/products/42`	Nur Änderungen	`200` / `204`
Löschen	`DELETE`	`/products/42`	—	`204`

Jenseits von CRUD

Nicht alles passt in CRUD. Für **Aktionen** gibt es zwei Muster:

- **Sub-Ressource:** `POST /orders/42/cancellations`
- **Controller-Ressource:** `POST /password-resets`

Workflow: Create / Update / Delete korrekt abbilden

Typische Zustandsänderungen und passende Responses

POST /products
201 Created + Location

PUT /products/42
200 OK oder 204

PATCH /products/42
200 OK oder 204

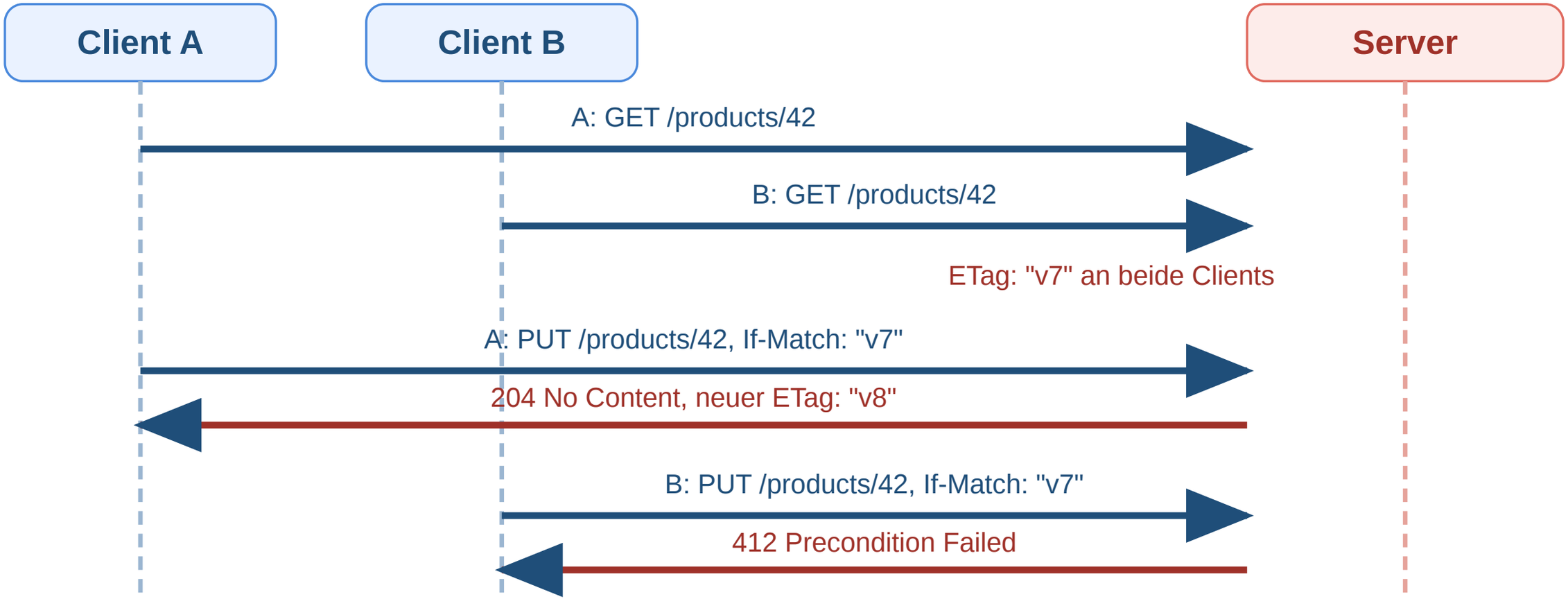
DELETE /products/42
204 No Content

POST erzeugt typischerweise eine neue Ressource, PUT/PATCH ändern eine vorhandene, DELETE entfernt sie.
Der Statuscode soll den fachlichen Effekt ausdrücken, nicht nur „irgendwie Erfolg“.

Operation	Gute Response	Warum?
Anlegen	`201 Created` + `Location`	Neue Ressource ist entstanden
Aktualisieren	`200` oder `204`	Ressource existierte bereits
Löschen	`204 No Content`	Erfolgreich, aber kein Body nötig

Workflow: Conditional PUT mit ETag und If-Match

Ziel: Verhindern von konfligierenden Updates. Das PUT wird nur ausgeführt, wenn die Ressource noch die erwartete Version hat.



Pagination, Filter und Sortierung

Query-Parameter beschreiben Auswahl, Reihenfolge und Ausschnitt einer Collection — nicht die Identität der Ressource selbst.

```
GET /products?page=2&per_page=20&sort=-price&category=electronics&q=kopfhörer
```

Parameter	Funktion	Beispiel
`page` / `per_page`	Seitenbasierte Pagination	`?page=3&per_page=25`
`cursor`	Cursor-basierte Pagination	`?cursor=eyJpZCI6NDJ9`
`sort`	Sortierung (`-` = absteigend)	`?sort=-created_at,name`
`filter`	Filterung	`?status=active&min_price=10`
`q`	Volltextsuche	`?q=kopfhörer`

Pagination in der Response

```
{
  "data": [{"id": 42, "name": "Funkkopfhörer", "price": 79.99}],
  "meta": { "total": 142, "page": 2, "per_page": 20 },
  "links": {
    "next": "/products?page=3&per_page=20",
    "prev": "/products?page=1&per_page=20"
  }
}
```

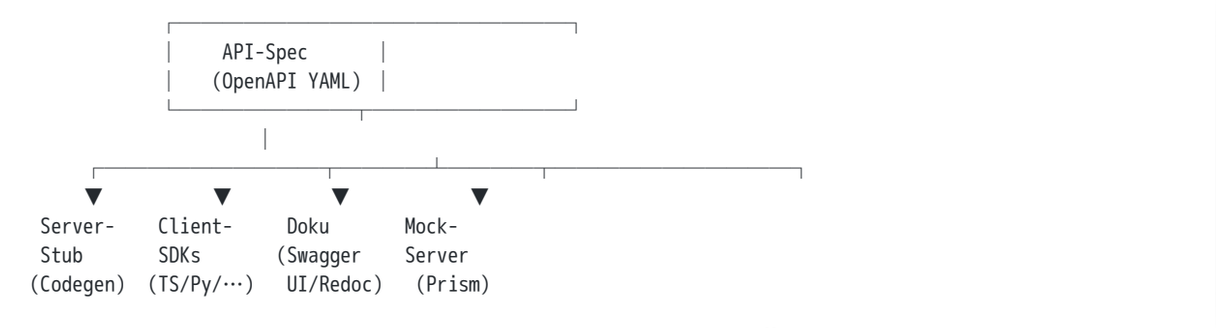
Links machen die API navigierbar — der Client muss keine URIs konstruieren.

Gute REST-APIs beginnen mit sauber geschnittenen Ressourcen. URI-Design ist eine **Modellierungsaufgabe**: Ressourcen identifizieren, Beziehungen als Hierarchie abbilden, Aktionen über

HTTP-Methoden ausdrücken. Die Entscheidung, wo eine Ressource in der Hierarchie steht, hängt vom Nutzungskontext ab — nicht von der Datenbankstruktur.

Die API als zentrales Design-Artefakt

Idee: Die API-Spezifikation ist nicht Beiwerk zur Implementierung, sondern das **zentrale Design-Artefakt**. Aus ihr werden alle anderen Bausteine synchronisiert.



Vorteil: Eine Quelle hält alles synchron. Spec-Änderung → Doku-Update, SDK-Update, neue Mock-Antworten, aktualisierte Test-Erwartungen.

API-First als Strategie — das Bezos-Mandat (Amazon, 2002):

> *„All teams will henceforth expose their data and functionality through service interfaces. […] There will be no other form of interprocess communication allowed: no direct linking, no direct reads of another team's data store, no shared-memory model, no back-doors whatsoever. […] All service interfaces, without exception, must be designed from the ground up to be **externalizable**."*

— Bezos-Memo, ~2002, öffentlich rekonstruiert durch Steve Yegges Plattform-Rant (2011).

> **Konsequenz:** Jedes Amazon-Team musste sich behandeln, als wäre es ein externer Anbieter. Diese Disziplin machte aus internen Diensten später **AWS** — die Infrastruktur war API-strukturiert und damit nach außen verkaufbar.

API-Design ist also nicht nur eine technische, sondern eine **organisatorische** Entscheidung. Spec-first zwingt Teams, Schnittstellen explizit zu klären, bevor Code entsteht.

API-Dokumentation mit OpenAPI

Beispiel — OpenAPI 3 Spec für `GET /products/{id}`:

```yaml
openapi: 3.0.3
info:
  title: Shop API
  version: 1.0.0
paths:
  /products/{id}:
    get:
      summary: Produkt abrufen
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
      responses:
        '200':
          description: Produkt gefunden
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Product'
        '404':
          description: Produkt nicht gefunden
```

Was die Spec definiert:

- **Endpunkte und Methoden** — `paths` listet alle URIs mit den jeweils erlaubten HTTP-Verben.
- **Parameter** — Pfad-, Query-, Header- und Cookie-Parameter mit Typ und ob `required`.
- **Request-/Response-Schemata** — Body-Strukturen via JSON Schema (`$ref` referenziert wiederverwendbare Komponenten unter `components/schemas`).
- **Statuscodes** — pro Antwort: welche Codes treten auf, mit welchem Body-Schema.

Was OpenAPI dadurch ermöglicht:

Vorteil	Erklärung
Maschinenlesbar	Codegen für Client-SDKs und Server-Stubs
Interaktive Docs	Swagger UI / Redoc rendert direkt aus YAML, mit „Try it out"
Contract Testing	Spec als Soll-Vorgabe für Property-/Konformitätstests
Single Source of Truth	Doku und Implementierung können nicht mehr auseinanderlaufen

> **OpenAPI ≠ Swagger.** Swagger ist die ursprüngliche Toolchain (heute Quasi-Synonym für die UI); der offene Standard heißt seit 2016 **OpenAPI Specification (OAS)** und wird von der Linux Foundation gepflegt.

OpenAPI-Tooling im Überblick

Die wichtigsten Werkzeuge

Werkzeug	Zweck
Swagger UI / **Redoc** / **Stoplight Elements**	Interaktive HTML-Doku aus der Spec; „Try it out"-Funktion ruft echte Endpunkte auf
openapi-generator / **Kiota** (Microsoft)	Code-Generierung: Client-SDKs (TypeScript, Python, Java, …) und Server-Stubs aus der Spec
Spectral	Linter — prüft Spec gegen Style-Regeln (Naming, Pflicht-Statuscodes, Beschreibungen, Versionspräfix, …)
Postman / **Bruno** / **Hurl**	API-Tests aus der Spec, sowohl manuell als auch in CI
Prism (Stoplight)	Mock-Server, der die Spec direkt als laufende Fake-API ausführt — Frontend kann gegen die Spec entwickeln, bevor das Backend existiert
schemathesis	Property-based Fuzz-Tests: generiert Requests aus der Spec und prüft Konformität der Responses

Best Practices für den Spec-Workflow

- **Contract-first** statt Code-first: Spec wird zuerst geschrieben und im Team reviewt, dann Code dazu. Vermeidet, dass interne Implementierungsdetails in die öffentliche API durchschlagen.
- **Spec lebt im Repo**, versioniert mit dem Code. Nicht in einem separaten Doku-Tool, das auseinanderlaufen kann.
- **Lint in CI**: Spectral als Pflicht-Check vor jedem Merge. Naming-Konsistenz, fehlende Beschreibungen, Pflicht-Statuscodes (`400`, `401`, `404`, `500`) erkennt der Linter automatisch.
- **Breaking-Change-Detection** in CI (z.B. `oasdiff`, `openapi-diff`): jeder PR wird gegen die letzte released Version verglichen; Breaking Changes erfordern Major-Versionssprung.
- **Mock-Server für Parallel-Entwicklung**: Frontend-Team kann mit Prism gegen die Spec entwickeln, während das Backend-Team implementiert.
- **Spec-driven Tests**: schemathesis o.ä. prüft im CI, dass die laufende API den Versprechen der Spec entspricht.

> Die Spec ist nicht *Doku der API*, sondern *Vertrag zur API*. Wer sie als generierten Output behandelt, verschenkt 80 % des Nutzens.

Wrap-Up

HTTP ist ein semantisches Protokoll, kein Datentransport. Methoden drücken Absicht aus, Statuscodes kommunizieren Ergebnisse, Header steuern Caching, Sessions und Sicherheit. Infrastruktur — Browser, Caches, CDNs, Load Balancer, Monitoring — reagiert auf diese Semantik ohne Anwendungswissen.

Was wir durchgegangen sind:

- **Headers + Status Codes:** Vokabular für Metadaten und Ergebnisse — `Content-Type`, `Authorization`, `2xx`/`3xx`/`4xx`/`5xx`.
- **Representations + Caching:** Eine Ressource, viele Repräsentationen; Content Negotiation; mehrstufiges Caching mit `Cache-Control` und `ETag`; Redirects und asynchrone Workflows.
- **Sessions + CORS:** Zustand trotz Zustandslosigkeit via Cookies; Same-Origin als Isolation; CORS als kontrollierte Lockerung für moderne SPAs.
- **Authentifizierung + Sicherheit:** TLS für den Transport; API Key / Session Cookie / OAuth 2.0 / JWT für unterschiedliche Auth-Szenarien.
- **REST:** Constraints (Statelessness, Cacheability, Uniform Interface) als Architekturidee, nicht als Marketing-Begriff.
- **Ressourcen + URI-Design + OpenAPI:** fachlich stabile Ressourcen, konsistentes Verb-Mapping, Spec-zentriertes Tooling.

> **Die wichtigste Botschaft:** Wer HTTP-Semantik korrekt nutzt, gewinnt Cachebarkeit, Skalierbarkeit, Sicherheit und Beobachtbarkeit fast geschenkt. Wer alles in `POST` mit `200 OK` und „Fehler-JSON im Body" packt, verliert genau diese Vorteile — und merkt es erst, wenn die Infrastruktur nicht mehr mitspielt.

> HTTP ist das **Interaktionsmodell des Webs**. REST nutzt dieses Modell konsequent für API-Design — Ressourcen statt Funktionsaufrufe, einheitliche Schnittstelle statt proprietärer Protokolle, Evolvierbarkeit durch Spec und Versionierung statt durch Hoffnung.

Wie es weitergeht: Die nächste Vorlesung zeigt, wie diese APIs **serverseitig in Java implementiert** werden — vom HTTP-Handler bis zur Spring-Boot-Anwendung mit Persistenz und Authentifizierung.